```
# ========== 文件: index.html ==========
<!DOCTYPE html>
<html lang="zh-CN">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>文件对比 · File Compare</title>
<link rel="stylesheet" href="lib/codemirror.css">
<link rel="stylesheet" href="styles.css">
</head>
<body>

<div class="app" id="appRoot">
<header class="topbar">
  <div class="tabs" id="tabs"></div>
  <button id="tabAdd" class="tab-add" type="button" title="新建对比窗口">+</button>
  <div class="brand" title="文件对比 · 离线纯前端">
  </div>
  <a class="tea-link" href="https://github.com/weifengit/compare-text" target="_blank"
    rel="noopener" title="开源项目 · 去 GitHub 项目页看看，欢迎 Star / 请作者喝茶">☕
    请作者喝茶</a>
</header>

<div class="body-row">
  <aside class="sidebar" id="sidebar" aria-label="工具栏">
    <div class="sb-title">对比源</div>
    <div class="src-block">
      <input id="srcPathInput" type="text" placeholder="点击选择对比源文件夹"
    title="点击弹出文件夹选择" spellcheck="false" readonly>
      <button id="srcLoadBtn" class="btn" title="加载该路径下的子文件夹与文件">加载</button>
    </div>
    <div class="src-cur" id="srcPathCur">未设置对比源</div>

    <div class="sb-title">忽略选项</div>
    <div role="group" aria-label="忽略选项">
      <label class="chip" title="比较时忽略大小写"><input type="checkbox" id="optIgnoreCase"
    checked>忽略大小写</label>
      <label class="chip" title="统一 CRLF / LF 行结束符"><input type="checkbox"
    id="optIgnoreEol" checked>忽略CRLF/LF</label>
      <label class="chip" title="忽略空k、Tab 等所有空白（含行内空格）"><input type="checkbox"
    id="optIgnoreWhitespace" checked>忽略空格</label>
      <label class="chip" title="像处理段落一样忽略换行位置差异"><input type="checkbox"
    id="optIgnoreNewline" checked>忽略换行</label>
      <label class="chip" title="全角括号、冒号等视为与半角相同"><input type="checkbox"
    id="optIgnoreWidth" checked>忽略全/半角</label>
      <label class="chip" title="忽略括号、引号、逗号、句号、分号、百分号等标点差异"><input
    type="checkbox" id="optIgnorePunct" checked>忽略标点</label>
    </div>

    <div class="sb-title">历史记录</div>
    <div class="history-list" id="historyList"></div>
    <button id="historyClear" class="h-clear" type="button" hidden>清空全部</button>
  </aside>

  <main class="main">
    <div class="filterbar" id="filterbar">
      <button id="sidebarToggle" class="btn fb-toggle" type="button" aria-expanded="true"
    aria-controls="sidebar" title="展开 / 收起侧边栏">≡</button>
      <label class="fb-item">子文件夹 <span class="cb" id="dirSel"
    title="对比源下的子文件夹（输入过滤，点击选择）" data-placeholder="搜索子文件夹…"></span></lab
    el>
      <label class="fb-item">原始文件 <span class="cb" id="fileSelL"
    title="左侧（原文）文件（输入过滤，点击选择）" data-placeholder="搜索原始文件…"></span></label
    >
```

```
      <button id="swapBtn" class="btn fb-swap" type="button"
    title="左右互换：调换左右两侧的文件与内容">⇄</button>
      <label class="fb-item">修改文件 <span class="cb" id="fileSelR"
    title="右侧（修改后）文件（输入过滤，点击选择）" data-placeholder="搜索修改文件…"></span></lab
    el>
      <div class="fb-actions">
        <button id="resetSplitBtn" class="btn" title="左右拖动调整列宽后
    点击恢复左右两区等宽">重置居中</button>
        <button id="tidyBtn" class="btn primary" title="将两侧文本中多余的空格、制表符
    换行符与空行统一整理为一行">修整</button>
        <button id="toggleEditorsBtn" class="btn" title="隐藏 /
    显示主区域2（编辑区）">隐藏编辑区</button>
        <button id="foldBtn" class="btn" title="折叠 / 展开连续相同的行">折叠相同行</button>
        <button id="clearBtn" class="btn" title="清空加入比较的内容（编辑器与标注区
    ">清除</button>
        <button id="exportReportBtn" class="btn" title="把当前对比结果导出为自包含 HTML
    报告（含逐页快照与差异标注）">导出报告</button>
        <button id="xlsxCfgBtn" class="btn" hidden title="配置并对比表格（Excel）：多
    Sheet、表头行范围、键列">表格对比配置</button>
      </div>
    </div>

    <div class="statusbar"><span id="status" role="status"
    aria-live="polite">在两侧粘贴文本即可自动对比</span><span id="stats"
    class="stats"></span></div>

    <div class="layout" id="layout">
      <div class="results" id="results"></div>
      <div class="resize-bar" id="resultsResize" title="拖动调整标注区域高度"><span
    class="resize-grip">⋮⋮</span></div>
      <section class="editors" id="editors">
        <div class="edpane"><div class="edlabel">原文</div><textarea id="leftEd"
    spellcheck="false"></textarea></div>
        <div class="edpane"><div class="edlabel">修改后</div><textarea id="rightEd"
    spellcheck="false"></textarea></div>
      </section>
      <div class="resize-bar" id="editorsResize" title="拖动调整编辑区与 PDF 区之间的距离"><span
    class="resize-grip">⋮⋮</span></div>
      <div class="pdfarea" id="pdfarea">
        <div class="pdf-controls">
          <button id="pdfAuto" class="btn active"
    title="自动缩放：适应宽度但不超过实际大小；Ctrl+滚轮 手动缩放比例">自动缩放</button>
          <button id="pdfFitW" class="btn" title="适应宽度">适应宽度</button>
          <button id="pdfFitP" class="btn" title="适应整页（宽高都适配）">适应页面</button>
          <span class="pdf-zoom" id="pdfZoomHint"></span>
          <span class="pdf-spacer"></span>
          <button id="pdfFullscreen" class="btn" title="隐藏上方区域，释放空间显示
    PDF">全屏</button>
        </div>
        <div class="pdf-panels">
          <div class="pdf-panel" id="pdfLeft" title="左侧文件"></div>
          <div class="pdf-panel" id="pdfRight" title="右侧文件"></div>
        </div>
      </div>
      <div class="vsplit" id="vsplit" title="左右拖动调整列宽（双击恢复等宽）"><span
    class="vsplit-grip">⋮</span></div>
    </div>
  </main>
</div>
</div>

<!-- 对比源文件夹选择弹层 -->
<div class="picker-mask" id="pickerMask" hidden>
  <div class="picker" role="dialog" aria-label="选择对比源文件夹">
```

```
    <div class="picker-head">
      <span>选择文件夹</span>
      <button id="pickCloseBtn" class="btn" type="button" title="关闭">×</button>
    </div>
    <div class="picker-nav">
      <button id="pickerUp" class="btn" type="button" title="上一级">⬆ 上级</button>
      <input id="pickerPath" type="text" placeholder="输入或粘贴绝对路径后回车"
    spellcheck="false">
      <button id="pickerGo" class="btn" type="button"
    title="加载输入框中的路径，浏览其子文件夹">加载</button>
    </div>
    <div class="picker-body" id="pickerBody"></div>
    <div class="picker-err hidden" id="pickerErr"></div>
    <div class="picker-foot">
      <button id="pickCancelBtn" class="btn" type="button">取消</button>
      <button id="pickOkBtn" class="btn primary" type="button">选择此文件夹</button>
    </div>
  </div>
</div>

<!-- 扫描版 PDF OCR 确认弹窗 -->
<div class="ocr-mask" id="ocrMask" hidden>
  <div class="ocr-modal" role="dialog" aria-label="扫描版 PDF 识别">
    <div class="ocr-title">检测到扫描版 PDF（无文字层）</div>
    <div class="ocr-desc">这份 PDF 无法直接提取文字，对比和差异标注需要先识别（OCR）
    是否启用本地 OCR 识别？识别结果会自动加入对比，并标注在原扫描页面上。</div>
    <div class="ocr-file" id="ocrFile"></div>
    <div class="ocr-hint">首次识别需加载模型（约 15MB，之后复用），全程离线。</div>
    <div class="ocr-progress" id="ocrProgress" hidden>
      <span class="spin"></span><span class="ocr-progress-text" id="ocrProgressText"></span>
    </div>
    <div class="ocr-err" id="ocrErr" hidden></div>
    <div class="ocr-actions">
      <button id="ocrSkipBtn" class="btn" type="button" title="跳过
    OCR，不识别这份扫描版">跳过</button>
      <button id="ocrGoBtn" class="btn primary" type="button">OCR 识别</button>
    </div>
  </div>
</div>
<!-- 表格（Excel）对比配置弹窗：多 Sheet 勾选 + 表头行范围 + 键列 + 预览 -->
<div class="xc-mask" id="xlsxCfgMask" hidden>
  <div class="xc-modal" role="dialog" aria-label="表格对比配置">
    <div class="xc-head">
      <span class="xc-head-title" id="xlsxCfgTitle">表格对比配置</span>
      <button id="xlsxCfgClose" class="btn" type="button" title="关闭">×</button>
    </div>
    <div class="xc-body" id="xlsxCfgBody"></div>
    <div class="xc-foot">
      <span class="xc-foot-hint">表头行支持多行（起始～结束）；键列可多选拼成复合键</span>
      <span class="xc-foot-spacer"></span>
      <button id="xlsxCfgCancel" class="btn" type="button">取消</button>
      <button id="xlsxCfgOk" class="btn primary" type="button">开始对比</button>
    </div>
  </div>
</div>

<script src="lib/pdf.min.js"></script>
<script src="lib/codemirror.min.js"></script>
<script src="lib/diff.min.js"></script>
<script src="lib/jszip.min.js"></script>
<script src="lib/exceljs.min.js"></script>
<script src="lib/mammoth.browser.min.js"></script>
<script src="lib/docx-preview.min.js"></script>
<script src="src/normalize.js"></script>
```

```
<script src="src/compute.js"></script>
<script src="src/source-api.js"></script>
<script src="src/picker.js"></script>
<script src="src/combobox.js"></script>
<script src="src/filterbar.js"></script>
<script src="src/pdfview.js"></script>
<script src="src/docxview.js"></script>
<script src="src/syncscroll.js"></script>
<script src="src/tabs.js"></script>
<script src="src/ocr.js"></script>
<script src="src/report.js"></script>
<script src="src/pipeline.js"></script>
<script src="src/xlsx-io.js"></script>
<script src="src/xlsx-compare.js"></script>
<script src="src/xlsx-report.js"></script>
<script src="src/xlsxflow.js"></script>
<script src="src/xlsxview.js"></script>
<script src="src/sheetview.js"></script>
<script src="src/xlsxconfig.js"></script>
<script src="src/app.js"></script>
<script src="src/external-links.js"></script>
</body>
</html>

# ========== 文件：plugins/dsh-compare-text/cordis.patch.yml ==========
# dsh-compare-text bundle patch: insert the plugin row into the profile tree.
#
# 现状（2026-10-07 调整）：本插件只保留**服务端半侧**——
#   index.js 在 DSH 启动时拉起 127.0.0.1:<port> 的 compare-text serve.js，
#   并注册同源 GET /compare-text-meta。
# 客户端里那两处界面入口（输入框/footer 浮层入口、dsh-worktable 项目卡片）
# 都已移除，见 lib/client.js 顶部说明；compare-text 的展示改由 dsh-tokensave
# 项目中心的「展示」按钮以 iframe 内嵌呈现（地址来自本项目 package.json
# 的 previewUrl，由 tokensave 的 project-info 路由读取）。
- insert:
    - id: dsh-compare-text
      name: dsh-compare-text
      config:
        # compare-text 静态服务器端口（仅 127.0.0.1；被占用则复用现有实例）。
        # 需与 compare-text 根 package.json 的 previewUrl 保持一致。
        port: 3180

// ========== 文件：plugins/dsh-compare-text/index.js ==========
/**
 * dsh-compare-text — 服务端半侧（server half）
 *
 * 职责：
 *  1. 在 DSH 启动时，自动在 127.0.0.1:<port> 启动 compare-text 自带的静态服务器
 *     （serve.js），仅本机可访问（HOST=127.0.0.1），不会对外网暴露。
 *  2. 在 DSH 的 webserver 上注册 GET /compare-text-meta，返回对比工具的实际
 *     URL，供浏览器侧（lib/client.js）的浮层 iframe 使用。
 *
 * 端口默认 3180；若该端口已被占用（比如你已经手动跑过 serve.js），插件不会
 * 重复启动，而是直接复用现有实例。
 */
import { spawn } from 'node:child_process';
import { connect } from 'node:net';
import { dirname, join } from 'node:path';
import { existsSync } from 'node:fs';
import { fileURLToPath } from 'node:url';

export const name = 'dsh-compare-text';
export const inject = ['webServer'];
```

```
/** 对比工具静态服务器监听端口（仅 127.0.0.1）。 */
const DEFAULT_PORT = 3180;

const __dirname = dirname(fileURLToPath(import.meta.url));

/** 项目根目录：优先取环境变量 COMPARE_TEXT_ROOT，否则按插件包所在位置推导（plugins/dsh-compare-te
    xt → 项目根）。 */
function resolveProjectRoot() {
  if (process.env.COMPARE_TEXT_ROOT && existsSync(process.env.COMPARE_TEXT_ROOT)) {
    return process.env.COMPARE_TEXT_ROOT;
  }
  const inferred = join(__dirname, '..', '..');
  if (existsSync(join(inferred, 'serve.js'))) return inferred;
  if (existsSync(join(process.cwd(), 'serve.js'))) return process.cwd();
  return inferred;
}

/** 探测 127.0.0.1:port 是否已有进程在监听。 */
function isPortOpen(port) {
  return new Promise((resolve) => {
    const socket = connect({ port, host: '127.0.0.1' });
    let settled = false;
    const done = (ok) => {
      if (settled) return;
      settled = true;
      socket.destroy();
      resolve(ok);
    };
    socket.once('connect', () => done(true));
    socket.once('error', () => done(false));
    socket.setTimeout(500, () => done(false));
  });
}

export function apply(ctx, config) {
  const port = Number(config?.port) || DEFAULT_PORT;
  const root = resolveProjectRoot();
  const servePath = join(root, 'serve.js');
  const url = `http://127.0.0.1:${port}/`;
  const log = (...args) => {
    try { ctx.logger?.info?.('[dsh-compare-text]', ...args); } catch { /* noop */ }
  };
  const warn = (...args) => {
    try { ctx.logger?.warn?.('[dsh-compare-text]', ...args); } catch {
    console.warn('[dsh-compare-text]', ...args); }
  };

  let child = null;
  let disposed = false;

  // 启动（或复用）对比工具的静态服务器；dispose 时杀掉由本插件拉起的进程。
  ctx.effect(() => {
    log(`starting compare-text server at ${url} (serve.js: ${servePath})`);
    if (!existsSync(servePath)) {
      warn(`serve.js not found at ${servePath}; set COMPARE_TEXT_ROOT to the project root.`);
      return;
    }
    isPortOpen(port).then((open) => {
      if (disposed) return;
      if (open) {
        log(`port ${port} already in use — reusing existing compare-text instance.`);
        return;
      }
      log(`spawning: node ${servePath} ${port}`);
```

```
      child = spawn(process.execPath, [servePath, String(port)], {
        cwd: root,
        env: { ...process.env, HOST: '127.0.0.1', PORT: String(port) },
        stdio: 'ignore',
        windowsHide: true,
      });
      child.on('error', (err) => warn(`failed to spawn serve.js: ${err.message}`));
      child.on('exit', (code, signal) => {
        if (!disposed) warn(`serve.js exited (code=${code}, signal=${signal}); the compare panel
    will fail to load.`);
        child = null;
      });
    });
    return () => {
      disposed = true;
      if (child) {
        try { child.kill(); } catch { /* noop */ }
        child = null;
      }
    };
  }, 'dsh-compare-text: spawn static server');

  // 供浏览器侧查询对比工具 URL（同源 /compare-text-meta，无 CORS 问题）。
  ctx.webServer.register({
    kind: 'exact',
    path: '/compare-text-meta',
    handler: async (_req, res) => {
      res.writeHead(200, {
        'Content-Type': 'application/json; charset=utf-8',
        'Cache-Control': 'no-store',
      });
      res.end(JSON.stringify({ ok: true, url, port }));
    },
  });
}

// ========== 文件：plugins/dsh-compare-text/lib/client.js ==========
/**
 * dsh-compare-text — 客户端半侧（Web GUI bundle）
 *
 * 现状：**本插件不再注册任何界面入口**。
 *
 * 背景（用户 2026-10-07 调整）：原来 compare-text 有两处界面入口——
 *   1）「文本对比」全屏浮层入口（输入框工具行 / 悬浮球 / dock / 侧边栏底部图标，由 config.entry
    选）；
 *   2）dsh-worktable「控制室」里的「文本对比」项目卡片（sidebar.worktable.project 子座位）。
 * 现在这两处都已**移除**：compare-text 的展示统一交给 **dsh-tokensave 项目中心**——
 * 在项目中心里点该项目卡片的「展示」按钮，即以 iframe 在 DSH 内呈现对比工具。
 *
 * 因此本客户端 bundle 只保留一个「空 apply」：
 *   - 继续满足 package.json 的 dsh.client 声明与 __ModuleLoader__ 握手（模块能正常加载）；
 *   - 不注册任何 slot，界面上不再出现任何 compare-text 入口/卡片。
 *
 * 服务端半侧（index.js）保持不动：仍负责在 DSH 启动时拉起 127.0.0.1:<port> 的
 * serve.js，并注册同源 GET /compare-text-meta——对比工具的实际地址由项目中心侧
 * 经项目的 devUrl 推导（见 compare-text 根 package.json 的 dsh.devUrl）。
 */
window.__ModuleLoader__.load({
  id: "dsh-compare-text",
  factory: (require) => {
    var module = { exports: {} };
    var exports = module.exports;
    Object.defineProperty(exports, Symbol.toStringTag, { value: "Module" });
```

```
    /** 无 UI 入口：加载即可，什么都不注册。 */
    function apply() {
      /* intentionally empty — compare-text 的展示已交给 dsh-tokensave 项目中心 */
    }

    exports.apply = apply;
    exports.inject = [];
    return module.exports;
  },
});

# ========== 文件：plugins/dsh-compare-text/package.json ==========
{
  "name": "dsh-compare-text",
  "private": true,
  "description": "DSH 本地插件（仅服务端半侧）：DSH 启动时拉起 compare-text 的
    serve.js（127.0.0.1:3180）并注册 /compare-text-meta；界面展示由 dsh-tokensave
    项目中心发起，本插件不再注册任何入口",
  "version": "0.3.0",
  "type": "module",
  "main": "index.js",
  "exports": {
    ".": "./index.js",
    "./client": "./lib/client.js",
    "./package.json": "./package.json"
  },
  "files": [
    "index.js",
    "lib/client.js",
    "cordis.patch.yml"
  ],
  "dsh": {
    "bundle": {
      "patch": "./cordis.patch.yml"
    },
    "client": {
      "platform": "web"
    }
  },
  "license": "Apache-2.0"
}

// ========== 文件：scripts/bump-version.js ==========
#!/usr/bin/env node
/**
 * bump-version.js — 统一版本号脚本（自动发布工作流与手动发布共用）。
 *
 * 同步更新 5 个文件中的版本号，保证处处一致：
 *   - package.json
 *   - package-lock.json（顶层 version + packages[""].version）
 *   - src-tauri/tauri.conf.json
 *   - src-tauri/Cargo.toml（[package] 段的 version）
 *   - src-tauri/Cargo.lock（仅根包 name = "app" 条目，勿动依赖版本）
 *
 * 用法：
 *   node scripts/bump-version.js current         # 仅打印当前版本
 *   node scripts/bump-version.js patch           # 1.2.0 -> 1.2.1（补丁号 +1）
 *   node scripts/bump-version.js minor           # 1.2.0 -> 1.3.0
 *   node scripts/bump-version.js major           # 1.2.0 -> 2.0.0
 *   node scripts/bump-version.js 1.3.0           # 显式指定具体版本号
 *   node scripts/bump-version.js patch --dry-run # 只打印不写文件
 *
 * 约定：新版本号打印到 stdout（供工作流 `NEW=$(node scripts/bump-version.js ...)` 捕获），
 * 其余提示信息一律输出到 stderr，避免污染 stdout。
```

```
 */
'use strict';
var fs = require('fs');
var path = require('path');

var ROOT = path.join(__dirname, '..');
var PKG = path.join(ROOT, 'package.json');
var LOCK = path.join(ROOT, 'package-lock.json');
var CONF = path.join(ROOT, 'src-tauri', 'tauri.conf.json');
var CTOML = path.join(ROOT, 'src-tauri', 'Cargo.toml');
var CLOCK = path.join(ROOT, 'src-tauri', 'Cargo.lock');

function readJson(p) { return JSON.parse(fs.readFileSync(p, 'utf8')); }
function writeJson(p, o) { fs.writeFileSync(p, JSON.stringify(o, null, 2) + '\n', 'utf8'); }

// ---------- 参数解析 ----------
var args = process.argv.slice(2);
var DRY = args.indexOf('--dry-run') !== -1;
args = args.filter(function (a) { return a !== '--dry-run'; });
var mode = args[0] || '';

function bump(from, arg) {
  var m = /^(\d+)\.(\d+)\.(\d+)$/.exec(from);
  if (!m) throw new Error('无法解析语义化版本号: ' + from);
  var ma = +m[1], mi = +m[2], pa = +m[3];
  if (/^\d+\.\d+\.\d+$/.test(arg)) return arg;      // 显式版本
  if (arg === 'patch') { pa++; }
  else if (arg === 'minor') { mi++; pa = 0; }
  else if (arg === 'major') { ma++; mi = 0; pa = 0; }
  else throw new Error('无效参数: ' + arg + ' (支持 patch|minor|major|X.Y.Z|current) ');
  return ma + '.' + mi + '.' + pa;
}

// ---------- 读取当前版本 (以 package.json 为基准) ----------
var current = readJson(PKG).version;

if (!mode) {
  console.error('用法: node scripts/bump-version.js <patch|minor|major|X.Y.Z|current>
    [--dry-run]');
  process.exit(2);
}
if (mode === 'current') { console.log(current); process.exit(0); }

var next = bump(current, mode);
if (next === current) { console.log(next); process.exit(0); }   // 幂等

// ---------- 生成 5 个文件的新内容 ----------
var updates = [];   // [相对路径, 新内容]

var pkg = readJson(PKG); pkg.version = next;
updates.push(['package.json', JSON.stringify(pkg, null, 2) + '\n']);

var lock = readJson(LOCK); lock.version = next;
if (lock.packages && lock.packages['']) lock.packages[''].version = next;
updates.push(['package-lock.json', JSON.stringify(lock, null, 2) + '\n']);

var conf = readJson(CONF); conf.version = next;
updates.push(['src-tauri/tauri.conf.json', JSON.stringify(conf, null, 2) + '\n']);

var toml = fs.readFileSync(CTOML, 'utf8');
var newToml = toml.replace(/^version\s*=\s*"[^"]*"/m, 'version = "' + next + '"');
if (newToml === toml) throw new Error('Cargo.toml 中未找到 [package] 的 version 行');
updates.push(['src-tauri/Cargo.toml', newToml]);
```

```
var clock = fs.readFileSync(CLOCK, 'utf8');
var re = new RegExp('(name = "app"\r?\nversion = )"' + current.replace(/\./g, '\\.') + '"');
if (!re.test(clock)) {
  console.error('警告: Cargo.lock 中未找到根包 name = "app" 条目，跳过该文件');
} else {
  updates.push(['src-tauri/Cargo.lock', clock.replace(re, '$1"' + next + '"')]);
}

// ---------- 写盘 / 预览 ----------
if (DRY) {
  console.error('[dry-run] 版本: ' + current + ' -> ' + next + '，将更新以下文件（未写入）:');
  updates.forEach(function (u) { console.error('  - ' + u[0]); });
} else {
  updates.forEach(function (u) { fs.writeFileSync(path.join(ROOT, u[0]), u[1], 'utf8'); });
  console.error('版本已更新: ' + current + ' -> ' + next);
  updates.forEach(function (u) { console.error('  已更新: ' + u[0]); });
}
console.log(next);   // stdout 只输出新版本号

// ========== 文件: scripts/install-dsh-plugin.ps1 ==========
<#
================================================================================
install-dsh-plugin.ps1 — 把 compare-text 的 DSH Skill + Plugin 离线装进
DeepSeek Harness (Windows PowerShell 5.1+ / PowerShell 7)。

用法:
  .\scripts\install-dsh-plugin.ps1                # 安装 skill(全局) + plugin
  .\scripts\install-dsh-plugin.ps1 -SkillOnly      # 只装 skill
  .\scripts\install-dsh-plugin.ps1 -PluginOnly     # 只装 plugin
  .\scripts\install-dsh-plugin.ps1 -Profile tui    # 指定 profile (默认 web)

全程本地文件操作，不发布 npm / GitHub，不联网。
================================================================================
#>
[CmdletBinding()]
param(
  [string]$Profile = "web",
  [switch]$SkillOnly,
  [switch]$PluginOnly
)

$ErrorActionPreference = "Stop"

# ---- 定位项目根（本脚本所在目录的上级） ----
$ScriptDir = Split-Path -Parent $MyInvocation.MyCommand.Path
$ProjectRoot = Split-Path -Parent $ScriptDir
$SkillSrc = Join-Path $ProjectRoot ".dsh\skills\compare-report"
$PluginDir = Join-Path $ProjectRoot "plugins\dsh-compare-text"

Write-Host "==> 项目根: $ProjectRoot"

$Mode = if ($SkillOnly) { "skill" } elseif ($PluginOnly) { "plugin" } else { "all" }

# ---- 前置检查 ----
function Check-Cmd($Name) {
  if (-not (Get-Command $Name -ErrorAction SilentlyContinue)) {
    Write-Host "✗ 缺少 $Name，请先安装（见 docs\dsh-offline-install.md 第 2 节）"
    -ForegroundColor Red
    exit 1
  }
}

if ($Mode -ne "skill") {
  Check-Cmd "node"
```

```
  Check-Cmd "pnpm"
  Check-Cmd "dsh"
}

# ---- 计算 DSH_HOME（与 dsh 一致：$DSH_HOME 优先，否则 %USERPROFILE%\.dsh） ----
$DshHome = if ($env:DSH_HOME) { $env:DSH_HOME } else { Join-Path $env:USERPROFILE ".dsh" }
Write-Host "==> DSH_HOME: $DshHome"

# ============ 1) Skill ============
if ($Mode -eq "all" -or $Mode -eq "skill") {
  Write-Host ""
  Write-Host "==> 安装 Skill: compare-report"
  if (-not (Test-Path (Join-Path $SkillSrc "SKILL.md"))) {
    Write-Host "✗ 找不到技能文件：$SkillSrc\SKILL.md" -ForegroundColor Red
    exit 1
  }
  New-Item -ItemType Directory -Force (Join-Path $DshHome "skills") | Out-Null
  Copy-Item -Recurse -Force $SkillSrc (Join-Path $DshHome "skills\compare-report")
  Write-Host "   ✔ 已复制到 $DshHome\skills\compare-report\SKILL.md"
  Write-Host "     （新开会话后技能列表应出现 compare-report；若会话工作目录在项目内
    本步其实可省）"
}

# ============ 2) Plugin ============
if ($Mode -eq "all" -or $Mode -eq "plugin") {
  Write-Host ""
  Write-Host "==> 安装 Plugin: dsh-compare-text -> profile '$Profile'"
  if (-not (Test-Path (Join-Path $PluginDir "package.json"))) {
    Write-Host "✗ 找不到插件包：$PluginDir\package.json" -ForegroundColor Red
    exit 1
  }
  # 在项目根执行，用相对路径让 dsh 以项目根为基准解析
  Push-Location $ProjectRoot
  try {
    & dsh plugin --profile $Profile add ".\plugins\dsh-compare-text"
    if ($LASTEXITCODE -ne 0) { throw "dsh plugin add 失败（退出码 $LASTEXITCODE）" }
  } finally {
    Pop-Location
  }
  Write-Host ""
  Write-Host "==> 验证（应看到 # == dsh-compare-text 层）"
  $dump = & dsh --profile $Profile --dump-config 2>&1 | Out-String
  if ($dump -match "dsh-compare-text") {
    Write-Host "   ✔ 插件已注册到 profile '$Profile'"
  } else {
    Write-Host "✗ 未在配置树里找到 dsh-compare-text，请检查上面 dsh plugin 的输出"
    -ForegroundColor Red
    exit 1
  }
}

# ============ 3) 收尾 ============
Write-Host ""
Write-Host "✅ 完成。下一步：" -ForegroundColor Green
Write-Host "   1) 重启 DSH：dsh web"
Write-Host "   2) 打开 http://127.0.0.1:3080 → 侧边栏「项目中心」→ compare-text 卡片
    →「展示」（需 dsh-tokensave）"
Write-Host "   3) 对话里说「对比这两个文件夹的合同，出报告」即可用 compare-report 技能"
Write-Host ""
Write-Host "卸载：dsh plugin --profile $Profile remove dsh-compare-text ; Remove-Item -Recurse
    -Force $DshHome\skills\compare-report"

// ========== 文件：scripts/install-dsh-plugin.sh ==========
#!/usr/bin/env bash
```

```
# ============================================================================
# install-dsh-plugin.sh — 把 compare-text 的 DSH Skill + Plugin 离线装进
# DeepSeek Harness (macOS / Linux)。
#
# 用法:
#   scripts/install-dsh-plugin.sh                # 安装 skill(全局) + plugin
#   scripts/install-dsh-plugin.sh --skill-only   # 只装 skill
#   scripts/install-dsh-plugin.sh --plugin-only  # 只装 plugin
#   scripts/install-dsh-plugin.sh --profile web  # 指定 profile (默认 web)
#
# 全程本地文件操作,不发布 npm / GitHub,不联网。
# ============================================================================
set -euo pipefail

# ---- 参数 ----
PROFILE="web"
MODE="all"          # all | skill | plugin
while [[ $# -gt 0 ]]; do
  case "$1" in
    --profile) PROFILE="$2"; shift 2 ;;
    --skill-only) MODE="skill"; shift ;;
    --plugin-only) MODE="plugin"; shift ;;
    -h|--help)
      sed -n '2,12p' "$0" | sed 's/^# \{0,1\}//'
      exit 0 ;;
    *) echo "未知参数: $1"; exit 2 ;;
  esac
done

# ---- 定位项目根(本脚本所在目录的上级) ----
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
SKILL_SRC="$PROJECT_ROOT/.dsh/skills/compare-report"
PLUGIN_DIR="$PROJECT_ROOT/plugins/dsh-compare-text"

echo "==> 项目根: $PROJECT_ROOT"

# ---- 前置检查 ----
check_cmd() { command -v "$1" >/dev/null 2>&1 || { echo "✗ 缺少 $1,请先安装(见
    docs/dsh-offline-install.md 第 2 节)"; exit 1; }; }

if [[ "$MODE" != "skill" ]]; then
  check_cmd node
  check_cmd pnpm
  check_cmd dsh
fi

# ---- 计算 DSH_HOME(与 dsh 一致:$DSH_HOME 优先,否则 ~/.dsh) ----
DSH_HOME="${DSH_HOME:-$HOME/.dsh}"
echo "==> DSH_HOME: $DSH_HOME"

# ============ 1) Skill ============
if [[ "$MODE" == "all" || "$MODE" == "skill" ]]; then
  echo ""
  echo "==> 安装 Skill: compare-report"
  if [[ ! -f "$SKILL_SRC/SKILL.md" ]]; then
    echo "✗ 找不到技能文件: $SKILL_SRC/SKILL.md"; exit 1
  fi
  mkdir -p "$DSH_HOME/skills"
  cp -R "$SKILL_SRC" "$DSH_HOME/skills/compare-report"
  echo "   ✔ 已复制到 $DSH_HOME/skills/compare-report/SKILL.md"
  echo "   (新开会话后技能列表应出现 compare-report;若会话工作目录在项目内,本步其实可省)"
fi
```

```
# ============ 2) Plugin ============
if [[ "$MODE" == "all" || "$MODE" == "plugin" ]]; then
  echo ""
  echo "==> 安装 Plugin: dsh-compare-text -> profile '$PROFILE'"
  if [[ ! -f "$PLUGIN_DIR/package.json" ]]; then
    echo "✗ 找不到插件包: $PLUGIN_DIR/package.json"; exit 1
  fi
  # 在项目根执行，用相对路径让 dsh 以项目根为基准解析
  ( cd "$PROJECT_ROOT" && dsh plugin --profile "$PROFILE" add "./plugins/dsh-compare-text" )
  echo ""
  echo "==> 验证（应看到 # == dsh-compare-text 层）"
  dsh --profile "$PROFILE" --dump-config | grep -i "compare" || {
    echo "✗ 未在配置树里找到 dsh-compare-text，请检查上面 dsh plugin 的输出"; exit 1; }
  echo "   ✔ 插件已注册到 profile '$PROFILE'"
fi

# ============ 3) 收尾 ============
echo ""
echo "✅ 完成。下一步："
echo "   1) 重启 DSH：dsh web"
echo "   2) 打开 http://127.0.0.1:3080 → 侧边栏「项目中心」→ compare-text 卡片 →「展示」（需
    dsh-tokensave）"
echo "   3) 对话里说「对比这两个文件夹的合同，出报告」即可用 compare-report 技能"
echo ""
echo "卸载：dsh plugin --profile $PROFILE remove dsh-compare-text && rm -rf
    $DSH_HOME/skills/compare-report"

// ========== 文件：scripts/package-offline-bundle.sh ==========
#!/usr/bin/env bash
# ================================================================================
# package-offline-bundle.sh — 打一个离线安装包，方便拷到新电脑（不公开安装）。
#
# 产物：dist-dsh-offline/compare-text-dsh-plugin-<日期>.tar.gz
# 内容：
#   - 整个项目（含 .dsh/skills、plugins/dsh-compare-text、serve.js、
#     tools/dsh-report.js、tools/render.js、lib/、src/、index.html、styles.css）
#   - scripts/ 安装/卸载脚本
#   - docs/dsh-offline-install.md 说明文档
#   - 一个 INSTALL.txt 快捷指引
#
# 用法：
#   scripts/package-offline-bundle.sh              # 打包到 dist-dsh-offline/
#   scripts/package-offline-bundle.sh -o /path/out # 指定输出目录
# ================================================================================
set -euo pipefail

OUT_DIR="${1:-dist-dsh-offline}"
while [[ $# -gt 0 ]]; do
  case "$1" in
    -o|--out) OUT_DIR="$2"; shift 2 ;;
    -h|--help) echo "用法: $0 [-o <输出目录>]"; exit 0 ;;
    *) OUT_DIR="$1"; shift ;;
  esac
done

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"

STAMP="$(date +%Y%m%d-%H%M%S)"
OUT_DIR_ABS="$(cd "$PROJECT_ROOT" && mkdir -p "$OUT_DIR" && cd "$OUT_DIR" && pwd)"
ARCHIVE="$OUT_DIR_ABS/compare-text-dsh-plugin-$STAMP.tar.gz"

echo "==> 打包项目（排除 node_modules / .git / 临时产物）..."
tar --exclude='node_modules' \
```

```
    --exclude='.git' \
    --exclude='.dsh-tmp' \
    --exclude='.spike-out' \
    --exclude='.probe-out3b' \
    --exclude='dist' \
    --exclude='src-tauri/target' \
    --exclude='demo' \
    -czf "$ARCHIVE" \
    -C "$PROJECT_ROOT" \
    .dsh/skills plugins serve.js tools lib src index.html styles.css \
    scripts docs/dsh-offline-install.md package.json README.md LICENSE 2>/dev/null

# 生成快捷指引
cat > "$OUT_DIR_ABS/INSTALL.txt" <<'EOF'
======================== 离线安装 compare-text 到 DeepSeek Harness ========================

【新电脑步骤】(已装 Node + pnpm + dsh 的情况下)

1. 解压本包到固定路径，例如:
   macOS:   ~/code/compare-text
   Windows: D:\code\compare-text

2. 安装 (在解压后的项目根目录执行):
   macOS / Linux:  bash scripts/install-dsh-plugin.sh
   Windows:        powershell -ExecutionPolicy Bypass -File .\scripts\install-dsh-plugin.ps1

   脚本会: ① 复制技能到 ~/.dsh/skills/compare-report
           ② 执行 dsh plugin --profile web add ./plugins/dsh-compare-text
           ③ 用 dsh --profile web --dump-config 验证

3. 重启 DSH: dsh web，打开 http://127.0.0.1:3080
   插件自动在 127.0.0.1:3180 拉起 serve.js = 插件 OK;
   界面展示: 进入会话 → 侧边栏「项目中心」→ compare-text 卡片 →「展示」(需 dsh-tokensave);
   对话里说「对比这两个文件夹的合同，出报告」= 技能 OK。

4. 卸载: bash scripts/uninstall-dsh-plugin.sh
         或 .\scripts\uninstall-dsh-plugin.ps1

详细文档见 docs/dsh-offline-install.md
============================================================================================
EOF

echo "==> 已生成:"
echo "    $ARCHIVE"
echo "    $OUT_DIR_ABS/INSTALL.txt"
echo ""
echo "把这两个文件拷到新电脑即可 (U盘 / 局域网 / 网盘)。"

// ========== 文件: scripts/uninstall-dsh-plugin.ps1 ==========
<#
============================================================================
uninstall-dsh-plugin.ps1 — 卸载 compare-text 的 DSH Skill + Plugin (Windows)

用法:
  .\scripts\uninstall-dsh-plugin.ps1                 # 卸载 skill(全局) + plugin
  .\scripts\uninstall-dsh-plugin.ps1 -Profile tui    # 指定 profile (默认 web)
============================================================================
#>
[CmdletBinding()]
param(
  [string]$Profile = "web"
)

$ErrorActionPreference = "Stop"
```

```
$DshHome = if ($env:DSH_HOME) { $env:DSH_HOME } else { Join-Path $env:USERPROFILE ".dsh" }

Write-Host "==> 卸载 Plugin: dsh-compare-text (profile '$Profile')"
if (Get-Command dsh -ErrorAction SilentlyContinue) {
  & dsh plugin --profile $Profile remove dsh-compare-text
} else {
  Write-Host "    (dsh 不在 PATH，跳过 plugin 卸载；可手动编辑 $DshHome\profiles\$Profile\package.
    json)"
}

Write-Host "==> 删除全局 Skill: compare-report"
$SkillPath = Join-Path $DshHome "skills\compare-report"
if (Test-Path $SkillPath) {
  Remove-Item -Recurse -Force $SkillPath
  Write-Host "  ✔ 已删除 $SkillPath"
} else {
  Write-Host "    (未找到，跳过)"
}

Write-Host "✅ 完成。重启 DSH 后生效。"

// ========== 文件: scripts/uninstall-dsh-plugin.sh ==========
#!/usr/bin/env bash
# ============================================================================
# uninstall-dsh-plugin.sh — 卸载 compare-text 的 DSH Skill + Plugin (macOS/Linux)
#
# 用法:
#   scripts/uninstall-dsh-plugin.sh                # 卸载 skill(全局) + plugin
#   scripts/uninstall-dsh-plugin.sh --profile web  # 指定 profile (默认 web)
# ============================================================================
set -euo pipefail

PROFILE="web"
while [[ $# -gt 0 ]]; do
  case "$1" in
    --profile) PROFILE="$2"; shift 2 ;;
    -h|--help) echo "用法: $0 [--profile <name>]"; exit 0 ;;
    *) echo "未知参数: $1"; exit 2 ;;
  esac
done

DSH_HOME="${DSH_HOME:-$HOME/.dsh}"

echo "==> 卸载 Plugin: dsh-compare-text (profile '$PROFILE')"
command -v dsh >/dev/null 2>&1 && dsh plugin --profile "$PROFILE" remove dsh-compare-text \
  || echo "    (dsh 不在 PATH，跳过 plugin 卸载；可手动编辑 $DSH_HOME/profiles/$PROFILE/package.js
    on)"

echo "==> 删除全局 Skill: compare-report"
if [[ -d "$DSH_HOME/skills/compare-report" ]]; then
  rm -rf "$DSH_HOME/skills/compare-report"
  echo "  ✔ 已删除 $DSH_HOME/skills/compare-report"
else
  echo "    (未找到，跳过)"
fi

echo "✅ 完成。重启 DSH 后生效。"

// ========== 文件: serve.js ==========
/**
 * serve.js — 零依赖本地静态服务器（离线可用，无需 npm install）。
 * 用法:
 *   node serve.js            # 监听 0.0.0.0:3000，局域网内其他人可访问
```

```
 *   node serve.js 8080       # 自定义端口
 *   HOST=127.0.0.1 node serve.js   # 仅本机可访问
 *   node serve.js build      # 重建 Tauri 打包用的前端目录 dist/（等价旧 build-web.js）
 *
 * 附加文件 API（供页面"对比源"读取本地文件系统，按绝对路径）：
 *   GET /api/list?path=<绝对路径>   → { ok, path, dirs:[name...], files:[{name,size,ext}] }
 *   GET /api/file?path=<绝对路径>   → 文件字节流
 *   GET  /api/ocr-cache?path=<绝对路径>   → OCR 结果缓存查询（{ ok, hit, text?, items? }）
 *   POST /api/ocr-cache             → 写 OCR 结果缓存（body={ path, text, items }）
 * OCR 缓存键 = sha1(path+size+mtime)，文件变化自动失效；缓存目录默认 ~/.compare-text/ocr-cache
 *  （可用环境变量 DSH_OCR_CACHE 覆盖），供扫描版 PDF 识别结果跨运行复用、避免重复 OCR。
 *
 * ⚠ 安全提示：API 不限制读取路径。监听 0.0.0.0 时局域网内任何人可枚举/读取本机任意文件，
 * 仅供本机/可信环境使用，建议 `HOST=127.0.0.1 node serve.js`。
 */
'use strict';
var http = require('http');
var fs = require('fs');
var path = require('path');
var os = require('os');
var crypto = require('crypto');

var PORT = parseInt(process.argv[2] || process.env.PORT || '3000', 10);
var HOST = process.argv[3] || process.env.HOST || '0.0.0.0';
var ROOT = __dirname;

/** 取本机局域网 IPv4 地址（供打印，方便分享给他人） */
var IS_WIN = process.platform === 'win32';

function isDriveRoot(p) {
  return IS_WIN && /^[A-Za-z]:[\\/]?$/.test(p);
}

/** 列出某目录下的子文件夹（带完整路径），按名称排序 */
function listDirs(p, cb) {
  fs.readdir(p, { withFileTypes: true }, function (err, entries) {
    if (err) return cb(err);
    var out = [];
    for (var i = 0; i < entries.length; i++) {
      var e = entries[i];
      if (!e.isDirectory()) continue;
      if (IS_WIN && (e.name === 'System Volume Information' || e.name === '$RECYCLE.BIN'))
    continue;
      out.push({ name: e.name, path: path.join(p, e.name) });
    }
    out.sort(function (a, b) { return a.name < b.name ? -1 : 1; });
    cb(null, out);
  });
}

/** Windows 盘符列表（fs.access 逐个字母探测，避免外调 WMI） */
function listDrives(cb) {
  if (!IS_WIN) return cb(null, [{ name: '/', path: '/' }]);
  var letters = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'.split('');
  var out = [];
  var idx = 0;
  (function step() {
    var L = letters[idx++];
    if (!L) { out.sort(function (a, b) { return a.name < b.name ? -1 : 1; }); return cb(null,
    out); }
    fs.access(L + ':\\', function (err) { if (!err) out.push({ name: L + ':', path: L + ':\\' });
    step(); });
  })();
}
```

```
/** 上一级目录；'' 表示无上级（盘符根 / 系统根），此时由 browse 回退到盘符列表 */
function parentOf(bp) {
  var resolved = path.resolve(bp);
  if (IS_WIN) {
    if (isDriveRoot(resolved)) return '';                                  // C:\ → 上级是盘符列表
    var dir = path.dirname(resolved);
    return dir.replace(/[\\/]+$/, '');
  }
  if (resolved === '/') return '';
  return path.dirname(resolved);
}

/** 目录浏览：供页面"对比源"文件夹选择弹层使用 */
function browse(bp, cb) {
  bp = String(bp || '').trim();
  if (bp === '' || bp === '/' || bp === '\\') {
    if (IS_WIN) listDrives(function (err, drives) {
      if (err) return cb(err);
      cb(null, { path: '', parent: '', kind: 'drives', dirs: drives });
    });
    else listDirs('/', function (err, ds) {
      if (err) return cb(err);
      cb(null, { path: '/', parent: '', kind: 'dir', dirs: ds });
    });
    return;
  }
  fs.stat(bp, function (err, st) {
    if (err || !st.isDirectory()) return cb(new Error('目录不存在：' + bp));
    var abs = path.resolve(bp);
    listDirs(abs, function (err2, ds) {
      if (err2) return cb(err2);
      cb(null, { path: abs, parent: parentOf(abs), kind: 'dir', dirs: ds });
    });
  });
}

/** 取本机局域网 IPv4 地址（供打印，方便分享给他人） */
function lanIPv4() {
  var nets = os.networkInterfaces();
  var out = [];
  Object.keys(nets).forEach(function (name) {
    (nets[name] || []).forEach(function (iface) {
      if (iface.family === 'IPv4' && !iface.internal) out.push(iface.address);
    });
  });
  return out;
}
var MIME = {
  '.html': 'text/html; charset=utf-8',
  '.js': 'text/javascript; charset=utf-8',
  '.mjs': 'text/javascript; charset=utf-8',
  '.css': 'text/css; charset=utf-8',
  '.json': 'application/json; charset=utf-8',
  '.png': 'image/png',
  '.svg': 'image/svg+xml',
  '.ico': 'image/x-icon',
  '.wasm': 'application/wasm',
  '.onnx': 'application/octet-stream',
  '.jpg': 'image/jpeg',
  '.jpeg': 'image/jpeg'
};
var API_MIME = {
  // 表格：前端用 XlsxFlow 走 readBytes/arrayBuffer 读它，这里给对 MIME 是为了
```

```
  // 万一被浏览器直接打开时按二进制处理，而不是当 text/plain 显示成乱码。
  '.xlsx': 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet',
  '.xlsm': 'application/vnd.ms-excel.sheet.macroEnabled.12',
  '.xls': 'application/vnd.ms-excel',
  '.pdf': 'application/pdf',
  '.docx': 'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
  '.txt': 'text/plain; charset=utf-8',
  '.text': 'text/plain; charset=utf-8',
  '.md': 'text/plain; charset=utf-8',
  '.csv': 'text/plain; charset=utf-8',
  '.json': 'application/json; charset=utf-8'
};

function sendJson(res, code, obj) {
  var body = JSON.stringify(obj);
  res.writeHead(code, { 'Content-Type': 'application/json; charset=utf-8', 'Content-Length':
    Buffer.byteLength(body) });
  res.end(body);
}

// ---------- OCR 结果磁盘缓存（扫描版 PDF 识别结果复用） ----------
// 缓存键 = sha1(绝对路径 + 文件大小 + mtimeMs)：文件一变键即变，天然失效。
// 缓存目录自动降级：$DSH_OCR_CACHE → ~/.compare-text/ocr-cache（首选，持久）→ os.tmpdir()
//  （写不进去时的兜底，如受限沙箱）。进程内解析一次，保证 GET/POST 用同一目录。
var ocrCacheDirResolved = null;
function ocrCacheDir() {
  if (ocrCacheDirResolved) return ocrCacheDirResolved;
  var cands = [];
  if (process.env.DSH_OCR_CACHE) cands.push(process.env.DSH_OCR_CACHE);
  cands.push(path.join(os.homedir(), '.compare-text', 'ocr-cache'));
  cands.push(path.join(os.tmpdir(), 'compare-text-ocr-cache'));
  for (var i = 0; i < cands.length; i++) {
    try {
      fs.mkdirSync(cands[i], { recursive: true });
      var probe = path.join(cands[i], '.probe-' + process.pid);
      fs.writeFileSync(probe, '1');
      fs.unlinkSync(probe);
      ocrCacheDirResolved = cands[i];
      try { fs.writeFileSync('/tmp/ocr-cache-debug.log', 'dir=' + cands[i] + '\n', { flag: 'a'
    }); } catch (e3) {}
      return ocrCacheDirResolved;
    } catch (e) { try { fs.writeFileSync('/tmp/ocr-cache-debug.log', 'cand fail: ' + cands[i] + '
    ' + e.message + '\n', { flag: 'a' }); } catch (e3) {} }
  }
  ocrCacheDirResolved = cands[cands.length - 1];
  return ocrCacheDirResolved;
}
function ocrCacheKey(absPath, st) {
  return crypto.createHash('sha1').update(absPath + '|' + st.size + '|' +
    st.mtimeMs).digest('hex');
}
function ocrCacheFile(absPath, st) {
  return path.join(ocrCacheDir(), ocrCacheKey(absPath, st) + '.json');
}

/** 读缓存：命中且结构合法 → { text, items }；未命中/损坏 → null（调用方回退 OCR） */
function ocrCacheRead(absPath, st, cb) {
  fs.readFile(ocrCacheFile(absPath, st), 'utf8', function (err, raw) {
    if (err) return cb(null);
    var obj = null;
    try { obj = JSON.parse(raw); } catch (e) { /* 损坏 */ }
    if (!obj || typeof obj.text !== 'string' || !obj.text || !Array.isArray(obj.items) ||
    !obj.items.length) {
      try { fs.unlinkSync(ocrCacheFile(absPath, st)); } catch (e2) { /* 忽略 */ }
```

```
      return cb(null);
    }
    cb({ text: obj.text, items: obj.items });
  });
}

/** 写缓存：目录已由 ocrCacheDir 确保存在，原子写临时文件后改名（避免半截 JSON
    被读到）。失败静默。 */
function ocrCacheWrite(absPath, st, text, items) {
  var file = ocrCacheFile(absPath, st), tmp = file + '.' + process.pid + '.tmp';
  try { fs.writeFileSync('/tmp/ocr-cache-debug.log', 'write entry file=' + file + ' textlen=' +
    (text && text.length) + '\n', { flag: 'a' }); } catch (e3) {}
  var payload = JSON.stringify({ path: absPath, size: st.size, mtimeMs: st.mtimeMs, cachedAt:
    Date.now(), text: text, items: items });
  fs.writeFile(tmp, payload, function (err) {
    if (err) { try { fs.writeFileSync('/tmp/ocr-cache-debug.log', 'write fail: ' + err.message +
    '\n', { flag: 'a' }); } catch (e3) {} return; }
    fs.rename(tmp, file, function () {});
  });
}

/** 处理 /api/* 请求（导出便于 test/api-test.js 直接调用） */
function handleApi(req, res) {
  var u = new URL(req.url, 'http://localhost');
  var q = u.searchParams;
  if (u.pathname === '/api/list') {
    var lp = q.get('path');
    if (!lp) return sendJson(res, 400, { ok: false, error: '缺少 path 参数' });
    fs.readdir(lp, { withFileTypes: true }, function (err, entries) {
      if (err) return sendJson(res, 400, { ok: false, error: String((err && err.code) || err) });
      var dirs = [], files = [];
      for (var i = 0; i < entries.length; i++) {
        var e = entries[i];
        if (e.isDirectory()) {
          dirs.push(e.name);
        } else if (e.isFile()) {
          var st = null;
          try { st = fs.statSync(path.join(lp, e.name)); } catch (e2) { /* 忽略 */ }
          files.push({ name: e.name, size: st ? st.size : 0, ext:
    path.extname(e.name).toLowerCase().replace(/^\./, '') });
        }
      }
      dirs.sort();
      files.sort(function (a, b) { return a.name < b.name ? -1 : 1; });
      sendJson(res, 200, { ok: true, path: lp, dirs: dirs, files: files });
    });
    return;
  }
  if (u.pathname === '/api/browse') {
    browse(q.get('path') || '', function (err, data) {
      if (err) return sendJson(res, 400, { ok: false, error: String((err && err.message) || err)
    });
      sendJson(res, 200, Object.assign({ ok: true }, data));
    });
    return;
  }
  if (u.pathname === '/api/file') {
    var fp = q.get('path');
    if (!fp) return sendJson(res, 400, { ok: false, error: '缺少 path 参数' });
    fs.stat(fp, function (err, st) {
      if (err || !st.isFile()) return sendJson(res, 404, { ok: false, error: '文件不存在' });
      var ext = path.extname(fp).toLowerCase();
      res.writeHead(200, { 'Content-Type': API_MIME[ext] || 'application/octet-stream',
    'Content-Length': st.size });
```

```
      fs.createReadStream(fp).on('error', function () { res.end(); }).pipe(res);
    });
    return;
  }
  // OCR 结果缓存：GET 查询 / POST 写入（body={ path, text, items }）
  // 服务端按 path+size+mtime 算键；命中返回 { ok:true, hit:true, text, items }，未命中 {
    ok:true, hit:false }
  if (u.pathname === '/api/ocr-cache') {
    if (req.method === 'POST') {
      try { fs.writeFileSync('/tmp/ocr-cache-debug.log', 'POST arrived cl=' +
    (req.headers['content-length'] || '?') + '\n', { flag: 'a' }); } catch (e3) {}
      var chunks = [];
      req.on('data', function (c) { chunks.push(c); });
      req.on('end', function () {
        var body = null;
        try { body = JSON.parse(Buffer.concat(chunks).toString('utf8')); } catch (e) { try {
    fs.writeFileSync('/tmp/ocr-cache-debug.log', 'body parse fail len=' +
    Buffer.concat(chunks).length + '\n', { flag: 'a' }); } catch (e3) {} }
        if (!body || typeof body.path !== 'string' || typeof body.text !== 'string' || !body.text
    || !Array.isArray(body.items) || !body.items.length) {
          try { fs.writeFileSync('/tmp/ocr-cache-debug.log', 'body invalid path=' + (body &&
    body.path) + ' text=' + (body && typeof body.text) + ' items=' + (body &&
    Array.isArray(body.items) ? body.items.length : 'n/a') + '\n', { flag: 'a' }); } catch (e3) {}
          return sendJson(res, 400, { ok: false, error: '请求体需要 { path, text, items }' });
        }
        fs.stat(body.path, function (err, st) {
          if (err || !st.isFile()) { try { fs.writeFileSync('/tmp/ocr-cache-debug.log', 'stat
    fail ' + err + '\n', { flag: 'a' }); } catch (e3) {} return sendJson(res, 404, { ok: false,
    error: '文件不存在' }); }
          ocrCacheWrite(body.path, st, body.text, body.items);
          sendJson(res, 200, { ok: true });
        });
      });
      return;
    }
    var oc = q.get('path');
    if (!oc) return sendJson(res, 400, { ok: false, error: '缺少 path 参数' });
    fs.stat(oc, function (err, st) {
      if (err || !st.isFile()) return sendJson(res, 404, { ok: false, error: '文件不存在' });
      ocrCacheRead(oc, st, function (hit) {
        if (!hit) return sendJson(res, 200, { ok: true, hit: false });
        sendJson(res, 200, { ok: true, hit: true, text: hit.text, items: hit.items, pages:
    hit.items.length });
      });
    });
    return;
  }
  sendJson(res, 404, { ok: false, error: '未知 API' });
}

function start() {
  var server = http.createServer(function (req, res) {
    if (req.url.indexOf('/api/') === 0) { handleApi(req, res); return; }
    var urlPath = decodeURIComponent(req.url.split('?')[0]);
    if (urlPath === '/') urlPath = '/index.html';
    var file = path.normalize(path.join(ROOT, urlPath));
    if (file.indexOf(ROOT) !== 0) { res.writeHead(403); res.end('forbidden'); return; }
    fs.readFile(file, function (err, buf) {
      if (err) { res.writeHead(404); res.end('not found'); return; }
      var ext = path.extname(file).toLowerCase();
      res.writeHead(200, { 'Content-Type': MIME[ext] || 'application/octet-stream' });
      res.end(buf);
    });
  });
```

```
  server.listen(PORT, HOST, function () {
    var urls = ['http://localhost:' + PORT];
    if (HOST === '0.0.0.0' || HOST === '::') {
      urls.push('http://' + HOST + ':' + PORT);
      lanIPv4().forEach(function (ip) { urls.push('http://' + ip + ':' + PORT); });
      console.log('⚠  注意：当前监听 0.0.0.0，/api 可读取任意绝对路径，建议 `HOST=127.0.0.1 node
    serve.js` 仅本机使用');
    } else {
      urls.push('http://' + HOST + ':' + PORT);
    }
    console.log('文本对比工具已启动：');
    urls.forEach(function (u) { console.log('  ' + u); });
  });
  return server;
}

/** 重建 Tauri 打包用的前端目录 dist/（由 tauri.conf.json 的 beforeBuildCommand
    自动调用）。用法：node serve.js build */
function buildWeb() {
  var DIST = path.join(ROOT, 'dist');
  var ENTRIES = ['index.html', 'styles.css', 'lib', 'src', 'models'];
  fs.rmSync(DIST, { recursive: true, force: true });
  fs.mkdirSync(DIST, { recursive: true });
  ENTRIES.forEach(function (name) {
    fs.cpSync(path.join(ROOT, name), path.join(DIST, name), { recursive: true });
  });
  console.log('[build] dist/ 已生成：' + ENTRIES.join(', '));
}

if (require.main === module) {
  if (process.argv[2] === 'build') buildWeb();
  else start();
}

module.exports = { handleApi: handleApi, start: start, MIME: MIME, API_MIME: API_MIME,
  ocrCacheDir: ocrCacheDir, ocrCacheKey: ocrCacheKey, ocrCacheRead: ocrCacheRead, ocrCacheWrite:
    ocrCacheWrite };

// ========== 文件：src-tauri/src/lib.rs ==========
// compare-text 桌面端 Rust 侧。
// 契约与 serve.js 的 /api/* 保持一致，供 src/source-api.js 在 Tauri 模式下调用：
//   api_list(path)      → { ok, path, dirs:[name], files:[{name,size,ext}] }
//   api_browse(path)    → { ok, path, parent, kind, dirs:[{name,path}] }
//   api_read_file(path) → 原始字节（前端转 Blob URL 给 pdf.js / fetch 使用）

use serde_json::json;

#[derive(serde::Serialize)]
struct FileInfo {
    name: String,
    size: u64,
    ext: String,
}

#[derive(serde::Serialize)]
struct DirInfo {
    name: String,
    path: String,
}

/// 列出目录下的子文件夹（带完整路径），按名称排序；Windows 下跳过系统保留目录（同 serve.js）。
fn list_dirs(p: &str, is_win: bool) -> std::io::Result<Vec<DirInfo>> {
    let mut out = Vec::new();
```

```
    for entry in std::fs::read_dir(p)? {
        let entry = entry?;
        let name = entry.file_name().to_string_lossy().into_owned();
        if !entry.file_type()?.is_dir() {
            continue;
        }
        if is_win && (name == "System Volume Information" || name == "$RECYCLE.BIN") {
            continue;
        }
        out.push(DirInfo {
            path: std::path::Path::new(p).join(&name).to_string_lossy().into_owned(),
            name,
        });
    }
    out.sort_by(|a, b| a.name.cmp(&b.name));
    Ok(out)
}

/// Windows 盘符列表（逐个字母探测存在的驱动器），供空路径浏览时返回；其他平台不可用。
fn list_drives() -> Vec<DirInfo> {
    let mut out = Vec::new();
    for l in 'A'..='Z' {
        let root = format!("{}:\\", l);
        if std::path::Path::new(&root).exists() {
            out.push(DirInfo { name: format!("{}:", l), path: root });
        }
    }
    out.sort_by(|a, b| a.name.cmp(&b.name));
    out
}

/// 上一级目录；'' 表示无上级（系统根 / 盘符根），此时由前端回落到根或盘符列表。
fn parent_of(resolved: &str, is_win: bool) -> String {
    if is_win {
        // 盘符根（C:\ / C:/ / C:）→ 无上级
        let b = resolved.as_bytes();
        let drive_root = b.len() == 2 && b[1] == b':'
            || b.len() == 3 && b[1] == b':' && (b[2] == b'\\' || b[2] == b'/');
        if drive_root {
            return String::new();
        }
    }
    if resolved == "/" {
        return String::new();
    }
    match std::path::Path::new(resolved).parent() {
        Some(parent) => parent.to_string_lossy().into_owned(),
        None => String::new(),
    }
}

/// 目录列表：同 serve.js 的 GET /api/list。
#[tauri::command]
fn api_list(path: String) -> Result<serde_json::Value, String> {
    let entries = std::fs::read_dir(&path).map_err(|e| format!("{}: {}", path, e))?;
    let mut dirs: Vec<String> = Vec::new();
    let mut files: Vec<FileInfo> = Vec::new();
    for entry in entries {
        let entry = entry.map_err(|e| e.to_string())?;
        let name = entry.file_name().to_string_lossy().into_owned();
        let ft = entry.file_type().map_err(|e| e.to_string())?;
        if ft.is_dir() {
            dirs.push(name);
        } else if ft.is_file() {
```

```
            let size = entry.metadata().map(|m| m.len()).unwrap_or(0);
            let ext = std::path::Path::new(&name)
                .extension()
                .map(|s| s.to_string_lossy().to_lowercase())
                .unwrap_or_default();
            files.push(FileInfo { name, size, ext });
        }
    }
    dirs.sort();
    files.sort_by(|a, b| a.name.cmp(&b.name));
    Ok(json!({ "ok": true, "path": path, "dirs": dirs, "files": files }))
}

/// 目录浏览：同 serve.js 的 GET /api/browse（文件夹选择弹层用）。
#[tauri::command]
fn api_browse(path: String) -> Result<serde_json::Value, String> {
    let is_win = std::env::consts::OS == "windows";
    let p = path.trim().to_string();
    if p.is_empty() || p == "/" || p == "\\" {
        if is_win {
            return Ok(json!({ "ok": true, "path": "", "parent": "", "kind": "drives", "dirs":
    list_drives() }));
        }
        let dirs = list_dirs("/", false).map_err(|e| e.to_string())?;
        return Ok(json!({ "ok": true, "path": "/", "parent": "", "kind": "dir", "dirs": dirs }));
    }
    let meta = std::fs::metadata(&p).map_err(|_| format!("目录不存在：{}", p))?;
    if !meta.is_dir() {
        return Err(format!("目录不存在：{}", p));
    }
    let abs = std::path::absolute(&p)
        .map_err(|e| e.to_string())?
        .to_string_lossy()
        .into_owned();
    let dirs = list_dirs(&abs, is_win).map_err(|e| e.to_string())?;
    let parent = parent_of(&abs, is_win);
    Ok(json!({ "ok": true, "path": abs, "parent": parent, "kind": "dir", "dirs": dirs }))
}

/// 读取文件字节：同 serve.js 的 GET /api/file（前端转 Blob URL）。
#[tauri::command]
fn api_read_file(path: String) -> Result<Vec<u8>, String> {
    let meta = std::fs::metadata(&path).map_err(|_| "文件不存在".to_string())?;
    if !meta.is_file() {
        return Err("文件不存在".to_string());
    }
    std::fs::read(&path).map_err(|e| e.to_string())
}

/// 写文本文件（UTF-8）：导出报告用。父目录不存在时自动创建。
#[tauri::command]
fn api_write_file(path: String, content: String) -> Result<(), String> {
    let p = std::path::Path::new(&path);
    if let Some(parent) = p.parent() {
        if !parent.as_os_str().is_empty() && !parent.exists() {
            std::fs::create_dir_all(parent).map_err(|e| format!("创建目录失败：{}", e))?;
        }
    }
    std::fs::write(p, content.as_bytes()).map_err(|e| format!("写入失败：{}", e))
}

/// 写二进制文件：Excel 报告导出用（xlsx 是二进制，不能走 api_write_file 的 UTF-8 字符串）。
/// 前端传 base64 字符串而非 number[]：JS 侧把字节数组序列化成 JSON 数组会让 1MB 文件膨胀到 4MB+，
/// 且 JSON 数字逐个解析在 IPC 上耗时差一个量级。base64 解码自己实现，不引第三方 crate。
```

```
#[tauri::command]
fn api_write_file_binary(path: String, base64: String) -> Result<(), String> {
    let data = decode_base64(&base64).ok_or_else(|| "base64
    解码失败（数据可能不完整）".to_string())?;
    let p = std::path::Path::new(&path);
    if let Some(parent) = p.parent() {
        if !parent.as_os_str().is_empty() && !parent.exists() {
            std::fs::create_dir_all(parent).map_err(|e| format!("创建目录失败: {}", e))?;
        }
    }
    std::fs::write(p, &data).map_err(|e| format!("写入失败: {}", e))
}

/// 标准 base64 解码（支持换行与 = 补位；非法字符返回 None）。
fn decode_base64(s: &str) -> Option<Vec<u8>> {
    let mut out = Vec::with_capacity(s.len() / 4 * 3);
    let mut buf: u32 = 0;
    let mut bits: u32 = 0;
    for b in s.bytes() {
        let v = match b {
            b'A'..=b'Z' => b - b'A',
            b'a'..=b'z' => b - b'a' + 26,
            b'0'..=b'9' => b - b'0' + 52,
            b'+' => 62,
            b'/' => 63,
            b'=' => break,           // 补位开始，后面不再有数据
            b'\n' | b'\r' | b' ' | b'\t' => continue,
            _ => return None,
        } as u32;
        buf = (buf << 6) | v;
        bits += 6;
        if bits >= 8 {
            bits -= 8;
            out.push(((buf >> bits) & 0xFF) as u8);
        }
    }
    Some(out)
}

/// 用系统默认浏览器打开外部链接（"请作者喝茶"等 target="_blank" 外链；WebView 不自动处理）。
#[tauri::command]
fn open_url(url: String) -> Result<(), String> {
    open::that(url).map_err(|e| e.to_string())
}

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    tauri::Builder::default()
        .invoke_handler(tauri::generate_handler![
            api_list,
            api_browse,
            api_read_file,
            api_write_file,
            api_write_file_binary,
            open_url
        ])
        .run(tauri::generate_context!())
        .expect("error while running tauri application");
}

// ========== 文件: src-tauri/src/main.rs ==========
// Prevents additional console window on Windows in release, DO NOT REMOVE!!
#![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]
```

```
fn main() {
  app_lib::run();
}

// ========== 文件: src/app.js ==========
/**
 * app.js — 界面层：多标签页、CodeMirror 编辑、选项、渲染（并排/流水）、同步滚动、
 * 折叠相同行、对比源/PDF 面板、历史记录（localStorage）、Worker 编排与降级。
 */
(function () {
  'use strict';

  // ---------- DOM ----------
  function $(id) { return document.getElementById(id); }
  var results = $('results');
  var statusEl = $('status');
  var statsEl = $('stats');
  var foldBtn = $('foldBtn');
  var toggleEditorsBtn = $('toggleEditorsBtn');
  var tidyBtn = $('tidyBtn');
  var resizeBar = $('resultsResize');
  var layoutEl = $('layout');
  var vsplitEl = $('vsplit');
  var historyList = $('historyList');
  var historyClearBtn = $('historyClear');
  var sidebarEl = $('sidebar');
  var sidebarToggle = $('sidebarToggle');
  var appEl = $('appRoot');
  var editorsEl = $('editors');
  var editorsResize = $('editorsResize');
  var clearBtn = $('clearBtn');
  var pdfFullscreen = $('pdfFullscreen');
  var srcPathInput = $('srcPathInput');
  var srcLoadBtn = $('srcLoadBtn');
  var srcPathCur = $('srcPathCur');
  var pdfareaEl = $('pdfarea');
  var swapBtn = $('swapBtn');
  var resetSplitBtn = $('resetSplitBtn');
  // 扫描版 OCR 弹窗
  var ocrMask = $('ocrMask');
  var ocrFile = $('ocrFile');
  var ocrProgress = $('ocrProgress');
  var ocrProgressText = $('ocrProgressText');
  var ocrErr = $('ocrErr');
  var ocrSkipBtn = $('ocrSkipBtn');
  var ocrGoBtn = $('ocrGoBtn');

  // ---------- 状态 ----------
  var OPT_MAP = {
    optIgnoreCase: 'ignoreCase',
    optIgnoreEol: 'ignoreEol',
    optIgnoreWhitespace: 'ignoreWhitespace',
    optIgnoreNewline: 'ignoreNewline',
    optIgnoreWidth: 'ignoreWidth',
    optIgnorePunct: 'ignorePunct'
  };
  // 差异计算、结果区渲染与主区域3 标注均在 src/pipeline.js（Pipeline），此处只保留界面状态
  var debounceTimer = null;
  var editorL, editorR;
  var HISTORY_KEY = 'diffchecker_history_v1';
  var RESIZE_KEY = 'diffchecker_results_h';
  var tidyBefore = null;     // 修整前的两侧文本快照（用于撤销）
  var resizeState = null;    // 标注区拖拽调整状态
  var switchSeq = 0;         // 单调递增：tab 切换 / 文件加载的陈旧异步丢弃令牌
```

```
var restoring = false;    // tab 恢复中：抑制 setValue 触发的防抖 compare
var lastCompareIsRestore = false;  // 本次 compare 是否由 tab 恢复发起（抑制历史记录）
var tabs = [];            // 多标签页：[{id, title, state}]
var activeTabId = null;
var tabSeq = 0;
var SIDEBAR_KEY = 'diffchecker_sidebar';
var sidebarOpen = true;      // 侧边栏实时状态（不读 classList.contains）
var sidebarPref = null;      // 持久化的用户偏好；null = 默认展开

// ---------- 工具 ----------
function escHtml(s) {
  return String(s).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g,
  '&gt;').replace(/"/g, '&quot;');
}
function baseName(p) {
  var parts = String(p || '').split(/[\\/]/);
  return parts[parts.length - 1] || '';
}
function stripExt(n) { return String(n || '').replace(/\.[A-Za-z0-9]{1,8}$/, ''); }
function abbrev(s, n) {
  s = String(s || '');
  return s.length > n ? s.slice(0, Math.max(1, n - 1)) + '…' : s;
}
function readOptions() {
  var o = {};
  for (var id in OPT_MAP) o[OPT_MAP[id]] = $(id).checked;
  return o;
}
function applyOptions(o) {
  for (var id in OPT_MAP) {
    var k = OPT_MAP[id];
    if (k in o) $(id).checked = !!o[k];
  }
}
function setBusy(b) {
  var r = Pipeline.getResult();
  if (b) { statusEl.innerHTML = '<span class="spin"></span>正在计算…'; }
  else if (!r) { statusEl.textContent = '在两侧粘贴文本即可自动对比'; }
  else if (r.error) { statusEl.textContent = '计算出错：' + r.error;
  statusEl.classList.add('bad'); }
  else { statusEl.textContent = '对比完成'; statusEl.classList.remove('bad'); }
}
function toast(msg, isErr) {
  var el = document.createElement('div');
  el.className = 'toast' + (isErr ? ' err' : '');
  el.textContent = msg;
  document.body.appendChild(el);
  setTimeout(function () { el.classList.add('out'); setTimeout(function () { el.remove(); },
  400); }, 1600);
}

// ---------- 历史记录 ----------
function loadHistory() {
  try {
    var raw = localStorage.getItem(HISTORY_KEY);
    return raw ? JSON.parse(raw) : [];
  } catch (e) { return []; }
}
function saveHistoryEntry() {
  var left = editorL.getValue(), right = editorR.getValue();
  if (!left && !right) return;
  try {
    var arr = loadHistory();
    if (arr.length && arr[0].left === left && arr[0].right === right
```

```
        && JSON.stringify(arr[0].options) === JSON.stringify(readOptions())) return;
    arr.unshift({ ts: Date.now(), left: left, right: right, options: readOptions() });
    arr = arr.slice(0, 100);
    localStorage.setItem(HISTORY_KEY, JSON.stringify(arr));
  } catch (e) { /* quota 等异常忽略 */ }
}

// ---------- 扫描版 OCR 弹窗（检测 → 确认 → 进度） ----------
var ocrSide = null;                  // 当前弹窗对应的侧（'L'|'R'）
var ocrBusy = false;                 // OCR 正在跑：弹窗转进度模式

function showOcrModal(side, absPath) {
  ocrSide = side;
  ocrBusy = false;
  ocrErr.hidden = true;
  ocrProgress.hidden = true;
  ocrSkipBtn.hidden = false;
  ocrSkipBtn.textContent = '跳过';
  ocrGoBtn.hidden = false;
  ocrGoBtn.textContent = 'OCR 识别';
  ocrFile.textContent = baseName(absPath);
  ocrMask.hidden = false;
}
function hideOcrModal() {
  ocrMask.hidden = true;
  ocrSide = null;
  ocrBusy = false;
}
function updateOcrProgress(side, info) {
  if (side !== ocrSide) return;
  if (info.phase === 'load') {
    // 首次加载模型：弹窗转进度
    ocrBusy = true;
    ocrProgress.hidden = false;
    ocrProgressText.textContent = '正在加载 OCR 模型（首次约 15MB，之后复用）…';
    ocrSkipBtn.hidden = true;
    ocrGoBtn.hidden = true;
  } else if (info.phase === 'page') {
    ocrProgress.hidden = false;
    ocrProgressText.textContent = '正在识别第 ' + info.page + ' / ' + info.total + ' 页…';
  } else if (info.phase === 'done') {
    ocrProgress.hidden = true;
  }
}
ocrSkipBtn.addEventListener('click', function () {
  if (ocrBusy) { Pipeline.cancelOcr(); hideOcrModal(); toast('已取消 OCR'); return; }
  hideOcrModal();
  toast('已跳过扫描版 PDF（无文字可对比）');
});
ocrGoBtn.addEventListener('click', function () {
  if (ocrBusy || !ocrSide) return;
  ocrBusy = true;
  ocrProgress.hidden = false;
  ocrProgressText.textContent = '正在准备…';
  ocrSkipBtn.hidden = true;
  ocrGoBtn.hidden = true;
  Pipeline.runOcr(ocrSide);
});

// ---------- 对比管线接线（差异计算 / 结果区渲染 / 主区域3 标注都在 Pipeline） ----------
Pipeline.init({
  resultsEl: results,
  statsEl: statsEl,
  // 编辑器文本与选项：管线只读写文本，不关心编辑器是谁
```

```
  getText: function (side) { return side === 'L' ? editorL.getValue() : editorR.getValue(); },
  setText: function (side, text) { if (side === 'L') editorL.setValue(text); else
  editorR.setValue(text); },
  getOptions: readOptions,
  setBusy: setBusy,
  toast: toast,
  // 一次对比落定：历史记录与 tab 标题（tab 恢复发起时不写历史）
  onResult: function (res, isRestore) {
    if (!isRestore) saveHistoryEntry();
    renderHistory();
    if (!isRestore) updateTabTitle();
  },
  // 结果区 DOM 重建后：重新绑定协同滚动（左右栏/流水栏都是新元素）
  onRendered: function () { rebindScrollSync(); },
  // 文件文本就绪：字段下拉 + 写入该侧编辑区
  onPanelText: function (side, text) {
    if (FilterBar.setFields) FilterBar.setFields(PdfView.segmentFields ?
  PdfView.segmentFields(text) : []);
    if (side === 'L') editorL.setValue(text); else editorR.setValue(text);
  },
  // 主区域3（PDF/Word 面板）显隐随面板加载结果重算
  onPanelsChanged: updatePdfArea,
  // 两侧皆空：状态栏回到初始文案
  onPlaceholder: function () { statusEl.textContent = '在两侧粘贴文本即可自动对比'; },
  // 扫描版 PDF（无文字层）检测到：弹出确认框，用户决定是否 OCR
  onScannedPdf: function (side, absPath) {
    showOcrModal(side, absPath);
  },
  // OCR 进度：加载模型 / 逐页识别
  onOcrProgress: function (side, info) {
    updateOcrProgress(side, info);
  },
  // 该侧 OCR 状态变化
  onOcrState: function (side, state) {
    if (state === 'running' || state === 'pending') return;   // 弹窗/进度条已由上面回调驱动
    if (state === 'done') {
      if (ocrMask && !ocrMask.hidden) hideOcrModal();
      statusEl.textContent = 'OCR 识别完成，已加入对比';
      statusEl.classList.remove('bad');
    } else if (state === 'failed') {
      if (ocrMask && !ocrMask.hidden) hideOcrModal();
    }
  }
});

// ---------- 事件 -------
function scheduleCompare() {
  if (restoring) return;                        // tab 恢复期间 setValue 触发的 change 不发对比
  Pipeline.updateMapMatches();                  // 编辑器文本变化 → PDF 表与编辑区是否仍同文本
  clearTimeout(debounceTimer);
  debounceTimer = setTimeout(function () { Pipeline.compare(); }, 400);
}

results.addEventListener('click', function (e) {
  var f = e.target.closest('.fold-row');
  if (f) {
    Pipeline.toggleFold(f.getAttribute('data-key'));
    var r = Pipeline.getResult();
    if (r && r.mode === 'grid') rerenderGridKeepScroll();
  }
});

// "修整"按钮单键切换：修整 ↔ 撤销修整（切换时改按钮文字与颜色）
function setTidyBtnMode(undo) {
```

```
  if (undo) {
    tidyBtn.textContent = '撤销修整';
    tidyBtn.classList.remove('primary');
    tidyBtn.classList.add('revert');
    tidyBtn.title = '撤销最后一次修整，恢复原文本';
  } else {
    tidyBtn.textContent = '修整';
    tidyBtn.classList.remove('revert');
    tidyBtn.classList.add('primary');
    tidyBtn.title = '将两侧文本中多余的空格、制表符、换行符与空行统一整理为一行';
  }
}
tidyBtn.addEventListener('click', function () {
  if (tidyBefore) {                        // 处于"撤销修整"态 → 撤销并恢复
    var b = tidyBefore;
    tidyBefore = null;
    editorL.setValue(b.left);
    editorR.setValue(b.right);
    setTidyBtnMode(false);
    toast('已撤销修整');
    setTimeout(function () { Pipeline.compare(); }, 0);
    return;
  }
  var left = editorL.getValue(), right = editorR.getValue();
  var vL = Norm.tidyText(left), vR = Norm.tidyText(right);
  if (vL === left && vR === right) { toast('文本无需修整'); return; }
  tidyBefore = { left: left, right: right };   // 快照，供撤销
  editorL.setValue(vL);
  editorR.setValue(vR);
  setTidyBtnMode(true);
  toast('已整理为一行，可撤销');
  setTimeout(function () { Pipeline.compare(); }, 0); // setValue 已触发
  change，此处兜底确保重算
});

// ---------- 历史记录 UI（内联于侧边栏，无需点击展开） ----------
function pad2(x) { return x < 10 ? '0' + x : '' + x; }
function renderHistory() {
  var arr = loadHistory();
  var html = arr.length ? '' : '<div class="h-empty">暂无历史记录</div>';
  for (var i = 0; i < arr.length; i++) {
    var it = arr[i];
    var d = new Date(it.ts);
    var time = pad2(d.getMonth() + 1) + '-' + pad2(d.getDate()) + ' '
             + pad2(d.getHours()) + ':' + pad2(d.getMinutes());
    var snippet = (it.left || '').replace(/\s+/g, ' ').slice(0, 24);
    html += '<div class="h-item">'
      + '<button class="h-restore" data-i="' + i + '">恢复</button>'
      + '<span class="h-time">' + time + '</span>'
      + '<button class="h-del" data-i="' + i + '" title="删除">×</button>'
      + '<span class="h-snippet" title="' + escHtml(snippet) + '">' + escHtml(snippet) +
  '</span></div>';
  }
  historyList.innerHTML = html;
  historyClearBtn.hidden = !arr.length;      // "清空全部"固定侧边栏最底部（index.html
  静态元素），无记录时隐藏
}
historyClearBtn.addEventListener('click', function () {
  try { localStorage.removeItem(HISTORY_KEY); } catch (e2) {}
  renderHistory();
});
historyList.addEventListener('click', function (e) {
  var t = e.target;
  if (t.classList.contains('h-restore')) {
```

```
    var it = loadHistory()[+t.getAttribute('data-i')];
    if (!it) return;
    editorL.setValue(it.left || '');
    editorR.setValue(it.right || '');
    applyOptions(it.options || {});
    Pipeline.compare();
  } else if (t.classList.contains('h-del')) {
    var arr = loadHistory();
    arr.splice(+t.getAttribute('data-i'), 1);
    try { localStorage.setItem(HISTORY_KEY, JSON.stringify(arr)); } catch (e2) {}
    renderHistory();
  }
});

// ---------- 标注区域高度可调 ----------
function clampResizeH(h) { return Math.max(110, Math.min(640, h)); }
function onResizeMove(ev) {
  if (!resizeState || resizeState.mode !== 'results') return;
  var y = ev.touches ? ev.touches[0].clientY : ev.clientY;
  results.style.height = clampResizeH(resizeState.base + (y - resizeState.y)) + 'px';
  queueRefitPage();
}
function onResizeUp() {
  if (!resizeState || resizeState.mode !== 'results') return;
  document.removeEventListener('mousemove', onResizeMove);
  document.removeEventListener('mouseup', onResizeUp);
  document.removeEventListener('touchmove', onResizeMove);
  document.removeEventListener('touchend', onResizeUp);
  try { localStorage.setItem(RESIZE_KEY, results.style.height); } catch (e) {}
  resizeState = null;
}
function beginResize(startY) {
  resizeState = { mode: 'results', y: startY, base: results.offsetHeight || 280 };
  document.addEventListener('mousemove', onResizeMove);
  document.addEventListener('mouseup', onResizeUp);
  document.addEventListener('touchmove', onResizeMove);
  document.addEventListener('touchend', onResizeUp);
}
resizeBar.addEventListener('mousedown', function (ev) { ev.preventDefault();
  beginResize(ev.clientY); });
resizeBar.addEventListener('touchstart', function (ev) {
  ev.preventDefault();
  var t = ev.touches && ev.touches[0];
  if (t) beginResize(t.clientY);
}, { passive: false });
try { var storedH = localStorage.getItem(RESIZE_KEY); if (storedH) results.style.height =
  storedH; } catch (e) {}

// ---------- 左右列宽调整 ----------
var SPLIT_KEY = 'diffchecker_split';
function setSplitPct(pct) {
  pct = Math.max(25, Math.min(75, pct));
  layoutEl.style.setProperty('--split', pct + '%');
}
function vsplitMove(ev) {
  if (!resizeState) return;
  var x = (ev.touches ? ev.touches[0].clientX : ev.clientX) - resizeState.rect.left;
  setSplitPct((x / resizeState.rect.width) * 100);
}
function vsplitUp() {
  if (!resizeState || resizeState.mode !== 'vsplit') return;
  vsplitEl.classList.remove('active');
  document.removeEventListener('mousemove', vsplitMove);
  document.removeEventListener('mouseup', vsplitUp);
```

```
  document.removeEventListener('touchmove', vsplitMove);
  document.removeEventListener('touchend', vsplitUp);
  try { localStorage.setItem(SPLIT_KEY, layoutEl.style.getPropertyValue('--split') || ''); }
  catch (e) {}
  if (PdfView.relayout) PdfView.relayout();   // 列宽变化后按当前缩放模式重排 PDF
  resizeState = null;
}
vsplitEl.addEventListener('mousedown', function (ev) {
  ev.preventDefault();
  vsplitEl.classList.add('active');
  resizeState = { mode: 'vsplit', rect: layoutEl.getBoundingClientRect() };
  document.addEventListener('mousemove', vsplitMove);
  document.addEventListener('mouseup', vsplitUp);
});
vsplitEl.addEventListener('touchstart', function (ev) {
  ev.preventDefault();
  vsplitEl.classList.add('active');
  resizeState = { mode: 'vsplit', rect: layoutEl.getBoundingClientRect() };
  document.addEventListener('touchmove', vsplitMove);
  document.addEventListener('touchend', vsplitUp);
}, { passive: false });
try {
  var savedSplit = localStorage.getItem(SPLIT_KEY);
  if (savedSplit && /^\d+(\.\d+)?%$/.test(savedSplit)) layoutEl.style.setProperty('--split',
  savedSplit);
} catch (e) {}

// ---------- 左右互换 / 重置居中 ----------
// 互换：编辑区文本、文件下拉、PDF 面板（连同其标注/文本项/原文缓存）整体对调；
// 修整快照同步对调，保证"撤销修整"仍恢复到对应侧。不重新读盘，互换即时完成。
function swapSides() {
  var pL = FilterBar.getSelected ? FilterBar.getSelected('L') : '';
  var pR = FilterBar.getSelected ? FilterBar.getSelected('R') : '';
  var tL = editorL.getValue(), tR = editorR.getValue();
  Pipeline.invalidateAll();                 // 作废旧侧在途加载，防止其回填覆盖互换结果
  editorL.setValue(tR);
  editorR.setValue(tL);
  if (tidyBefore) { var tb = tidyBefore.left; tidyBefore.left = tidyBefore.right;
  tidyBefore.right = tb; }
  if (FilterBar.selectFile) { FilterBar.selectFile('L', pR, true); FilterBar.selectFile('R',
  pL, true); }
  if (PdfView.swap) PdfView.swap();
  if (DocxView.swap) DocxView.swap();
  Pipeline.resetCharMaps();                 // PDF 互换：作废旧字符表，待重算
  updateTabTitle();
  toast('已左右互换');
}
if (swapBtn) swapBtn.addEventListener('click', swapSides);
// 重置居中：列宽恢复 50%（双击拖动条同效），PDF 按新列宽重排
function resetSplit() {
  setSplitPct(50);
  try { localStorage.removeItem(SPLIT_KEY); } catch (e) {}
  if (PdfView.relayout) PdfView.relayout();
  toast('已恢复左右等宽');
}
if (resetSplitBtn) resetSplitBtn.addEventListener('click', resetSplit);
vsplitEl.addEventListener('dblclick', resetSplit);

// ---------- 清除 ----------
clearBtn.addEventListener('click', function () {
  editorL.setValue('');
  editorR.setValue('');
  tidyBefore = null;
  setTidyBtnMode(false);
```

```
  height: 100%;
  border-radius: 2px;
  background: transparent;
  transition: background .15s;
}
.vsplit:hover::after, .vsplit.active::after { background: var(--accent); }
.vsplit-grip {
  color: #b6c2cf;
  font-size: 12px;
  position: absolute;
  pointer-events: none;
}
.grid {
  display: grid;
  grid-template-columns: var(--split) 1fr;
  grid-template-rows: auto 1fr;
  height: 100%;
}
.grid > * { min-width: 0; }
.colhead {
  padding: 7px 10px;
  font-weight: 600;
  background: #fafbfc;
  border-bottom: 1px solid var(--border);
  position: sticky;
  top: 0;
  z-index: 2;
}
.colhead + .colhead { border-left: 2px solid #ccd5e0; }
/* 原文栏 / 修改后栏：颜色底纹区分 */
.grid > .colhead:nth-child(1) { background: #eef4ff; }
.grid > .colhead:nth-child(2) { background: #fff3dd; }
.grid > .grid-body:nth-child(3) .row .ln { background: #eef4ff; }
.grid > .grid-body:nth-child(4) .row .ln { background: #fff3dd; }
/* 行号列底色延伸覆盖到整列最底部（容器级渐变，行内由 .ln 补齐） */
.grid > .grid-body:nth-child(3) {
  background-image: linear-gradient(90deg, #eef4ff 0, #eef4ff 3.2em, transparent 3.2em);
}
.grid > .grid-body:nth-child(4) {
  background-image: linear-gradient(90deg, #fff3dd 0, #fff3dd 3.2em, transparent 3.2em);
}
.colhead em { font-weight: 400; color: var(--muted); margin-left: 6px; font-style: normal; }
.grid-body {
  overflow: auto;
  overflow-anchor: none;   /* 同上：禁原生滚动锚定，避免与协同滚动程序化写回相互拉扯 */
  font-family: var(--mono);
  font-size: 13px;
  line-height: 1.55;
  position: relative;
}
.grid-body + .grid-body { border-left: 2px solid #ccd5e0; }

.row {
  display: flex;
  align-items: stretch;
  width: 100%;
  white-space: pre-wrap;
  word-break: break-all;
}
.row .ln {
  flex: 0 0 auto;
  width: 3.2em;
  padding: 0 6px 0 4px;
  text-align: right;
```

```
  color: var(--line-num);
  background: #f8fafc;
  border-right: 1px solid var(--border);
  user-select: none;
  /* 行号列吸附左侧，横向滚动时保持可见；随行高自动拉伸 */
  position: sticky;
  left: 0;
  z-index: 2;
}
.row .content {
  flex: 1 1 auto;
  min-width: 0;
  padding-right: 16px;
}
.row.same { background: #fff; }
.row.remove { background: var(--del-bg); }
.row.add { background: var(--add-bg); }
.row.change-l { background: var(--del-bg); }
.row.change-r { background: var(--add-bg); }
.row.empty { height: 1.55em; }

.hl { border-radius: 2px; padding: 0 1px; }
.hl.rm  { background: var(--del-strong); color: var(--del-text); }
.hl.ad  { background: var(--add-strong); color: var(--add-text); }
.hl.eq  { background: transparent; }

/* 折叠条 */
.fold-row { min-width: 100%; width: max-content; max-width: 100%; padding: 1px 8px 1px 4px;
    cursor: pointer; }
.fold-bar {
  margin: 2px 0;
  border: 1px dashed #b9c4d0;
  border-radius: 6px;
  background: var(--fold-bg);
  color: #34527a;
  text-align: center;
  font-size: 12px;
  line-height: 2;
  user-select: none;
}
.fold-bar:hover { background: #e2ebf7; }

.placeholder {
  height: 100%;
  display: flex;
  align-items: center;
  justify-content: center;
  color: var(--muted);
  font-size: 14px;
}

/* 忽略换行的流水视图（skipped 提示亦复用此容器） */
.inline-body {
  height: 100%;
  overflow: auto;
  font-family: var(--mono);
  font-size: 13px;
  line-height: 1.55;
}

/* 历史记录：内联显示在左侧边栏（"历史记录"标题之后，无需点击展开） */
/* 历史记录：列表动态撑满侧边栏剩余高度（不再限死 40vh），"清空全部"固定最底部 */
.history-list { flex: 1 1 auto; min-height: 60px; overflow: auto; margin-top: 4px; }
.history-list .h-item { flex-wrap: wrap; }
```

```
.history-list .h-snippet { width: 100%; order: 4; }
.hidden { display: none; }
#status.bad { color: var(--del-text); }
.h-item {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 7px 6px;
  border-bottom: 1px solid #f0f2f5;
}
.h-item:last-child { border-bottom: none; }
.h-restore, .h-del {
  border: 1px solid var(--border);
  background: #fff;
  border-radius: 6px;
  padding: 2px 8px;
  cursor: pointer;
  flex: 0 0 auto;
}
.h-restore:hover { border-color: var(--accent); color: var(--accent); }
.h-del { color: #c0392b; }
.h-time { color: var(--muted); font-size: 11.5px; flex: 0 0 auto; }
.h-snippet { overflow: hidden; text-overflow: ellipsis; white-space: nowrap; flex: 1 1 auto;
    color: var(--txt); }
.h-clear { flex: 0 0 auto; width: 100%; margin-top: 6px; padding: 5px; border: 1px solid
    var(--border); border-radius: 6px; background: #fff; cursor: pointer; color: var(--muted); }
.h-clear:hover { color: var(--del-text); }
.h-empty { padding: 14px; text-align: center; color: var(--muted); }

/* 文件夹选择弹层 */
.picker-mask {
  position: fixed;
  inset: 0;
  z-index: 80;
  background: rgba(20, 28, 38, .45);
  display: flex;
  align-items: center;
  justify-content: center;
}
.picker-mask[hidden] { display: none; }
.picker-mask.busy .picker-body { opacity: .55; pointer-events: none; }
.picker {
  width: min(560px, 92vw);
  height: min(520px, 80vh);   /* 固定高度：不随文件夹内容多少而变化 */
  background: var(--panel);
  border: 1px solid var(--border);
  border-radius: 10px;
  box-shadow: 0 18px 50px rgba(0, 0, 0, .3);
  display: flex;
  flex-direction: column;
  overflow: hidden;
}
.picker-head {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 10px 14px;
  border-bottom: 1px solid var(--border);
  font-weight: 600;
}
.picker-head .btn { border: none; font-size: 15px; padding: 2px 8px; line-height: 1; }
.picker-nav {
  display: flex;
  gap: 8px;
```

```
  padding: 10px 14px;
  border-bottom: 1px solid var(--border);
}
.picker-nav .btn { flex: 0 0 auto; margin: 0; }
.picker-nav input {
  flex: 1;
  min-width: 0;
  padding: 4px 8px;
  border: 1px solid var(--border);
  border-radius: 6px;
  font: inherit;
  color: var(--txt);
}
.picker-nav input:focus { outline: none; border-color: var(--accent); }
.picker-body {
  overflow: auto;
  padding: 4px 8px 10px;
  flex: 1 1 auto;        /* 填满弹层剩余高度，总高保持固定 */
  min-height: 0;
  font-family: var(--mono);
  font-size: 12.5px;
}
.picker-cur {
  padding: 8px 8px 6px;
  color: var(--muted);
  font-size: 11.5px;
  border-bottom: 1px dashed var(--border);
  margin-bottom: 4px;
  word-break: break-all;
}
.picker-row {
  padding: 5px 10px;
  border-radius: 6px;
  cursor: pointer;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  user-select: none;
}
.picker-row::before { content: '📁 '; opacity: .8; }
.picker-row:hover { background: #f0f4fa; }
.picker-row.active { background: #e2ebf7; color: var(--accent); }
.picker-empty { padding: 22px 10px; text-align: center; color: var(--muted); }
.picker-err { padding: 6px 14px 0; color: var(--del-text); font-size: 12px; }
.picker-err.hidden { display: none; }
.picker-foot {
  display: flex;
  justify-content: flex-end;
  gap: 8px;
  padding: 10px 14px;
  border-top: 1px solid var(--border);
  background: #fafbfc;
}
.picker-foot .btn { margin: 0; }

/* 提示条 */
.toast {
  position: fixed;
  left: 50%;
  bottom: 28px;
  transform: translateX(-50%);
  background: #22303f;
  color: #fff;
  padding: 8px 16px;
```

```
  border-radius: 8px;
  font-size: 12.5px;
  box-shadow: 0 6px 18px rgba(0,0,0,.25);
  z-index: 99;
  opacity: 1;
  transition: opacity .35s ease;
}
.toast.out { opacity: 0; }
.toast.err { background: #7a2233; }

.spin { display: inline-block; width: .9em; height: .9em; margin-right: 6px;
  border: 2px solid rgba(31,111,235,.25); border-top-color: var(--accent);
  border-radius: 50%; animation: rot .7s linear infinite; vertical-align: -1px; }
@keyframes rot { to { transform: rotate(360deg); } }

@media (max-width: 820px) {
  .editors { grid-template-columns: 1fr; height: auto; }
  .editors .edpane { height: 220px; }
  .pdf-panels { grid-template-columns: 1fr; grid-template-rows: minmax(220px, 1fr) minmax(220px,
    1fr); }
  .pdf-panel + .pdf-panel { border-left: none; border-top: 2px solid #ccd5e0; }
  .vsplit { display: none; }
}

/* ============================================================
 * 侧边栏高度约束修复
 * 症状：历史记录多时侧边栏被撑高，"清空全部"被推到视口之外。
 * 根因：侧边栏没有独立高度约束，历史列表没有自身滚动。
 * ============================================================ */

/* 页面根容器：占满视口、按列布局、禁止被子内容撑高 */
#appRoot {
  height: 100vh;
  display: flex;
  flex-direction: column;
  overflow: hidden;              /* ★ 关键：超出视口的部分不撑高页面 */
}

/* 历史列表：吃掉剩余空间，自身滚动，绝不撑高侧边栏 */
#historyList {
  flex: 1 1 0;                   /* ★ 关键：分配剩余空间 */
  min-height: 0;                 /* ★ 关键：允许缩到比内容小 */
  overflow-y: auto;              /* ★ 关键：历史多时自身出滚动条 */
  overflow-x: hidden;
}

/* 清空按钮：固定在侧边栏可见底部，不被列表挤出 */
#historyClear {
  flex: 0 0 auto;                /* ★ 关键：不收缩、不增长 */
  margin-top: auto;              /* 兜底：列表为空时也贴底 */
}

/* 侧边栏里的其他固定项（标题、切换按钮等）也不要被压缩 */
#sidebarToggle,
#sidebar > h3,
#sidebar > .sidebar-title {
  flex: 0 0 auto;
}

/* ============================================================
 * 扫描版 PDF OCR 确认弹窗与进度
 * ============================================================ */
.ocr-mask {
  position: fixed;
```

```
  inset: 0;
  z-index: 90;
  background: rgba(20, 28, 38, .45);
  display: flex;
  align-items: center;
  justify-content: center;
}
.ocr-mask[hidden] { display: none; }
.ocr-modal {
  width: min(480px, 92vw);
  background: var(--panel);
  border: 1px solid var(--border);
  border-radius: 10px;
  box-shadow: 0 18px 50px rgba(0, 0, 0, .3);
  padding: 16px 18px;
  font-size: 13px;
}
.ocr-title { font-weight: 600; font-size: 14.5px; margin-bottom: 8px; }
.ocr-desc { color: var(--muted); line-height: 1.55; word-break: break-all; margin-bottom: 6px; }
.ocr-file { font-family: var(--mono); font-size: 12px; color: var(--txt); word-break: break-all;
    margin-bottom: 10px; }
.ocr-hint { color: var(--muted); font-size: 11.5px; line-height: 1.5; }
.ocr-progress { display: flex; align-items: center; gap: 8px; margin: 10px 0 4px; }
.ocr-progress[hidden] { display: none; }
.ocr-progress-text { color: var(--muted); font-size: 12px; flex: 1 1 auto; }
.ocr-actions { display: flex; justify-content: flex-end; gap: 8px; margin-top: 14px; }
.ocr-actions .btn { margin: 0; }
.ocr-err { color: var(--del-text); font-size: 12px; margin-top: 6px; word-break: break-all; }
.ocr-err[hidden] { display: none; }
/* ================================================================================
   表格（Excel）对比
   配色与差异语义沿用文本区：红=删除/原文、绿=新增、琥珀=修改。
   ================================================================================ */

/* 主区域1：三视图（全部 / 新增 / 修改 / 删除） */
.xv {
  font-size: 12.5px;
  /* 结果区（.results）是固定高度 + overflow:hidden 的；三视图必须自己撑满并滚动，
     否则内容一多就被裁掉（.xv 自身不留 padding，改由内层分区各自控制，避免滚动条贴边）。 */
  height: 100%;
  display: flex;
  flex-direction: column;
  min-height: 0;
}
.xv-sheetbar {
  display: flex; flex-wrap: wrap; gap: 4px;
  padding: 8px 10px 8px; border-bottom: 1px solid var(--border);
  flex: 0 0 auto;          /* 不参与伸缩：滚动的是下面的行列表，不是它 */
}
.xv-sheet {
  font: inherit; font-size: 12px; padding: 3px 10px; cursor: pointer;
  border: 1px solid var(--border); background: var(--panel); color: var(--txt);
  border-radius: 5px; display: inline-flex; align-items: center; gap: 6px;
}
.xv-sheet:hover { background: var(--fold-bg); }
.xv-sheet.active { border-color: var(--accent); color: var(--accent); background: #eef4ff;
    font-weight: 600; }
.xv-sheet.nodiff { color: var(--muted); }
.xv-badge {
  font-size: 10.5px; min-width: 18px; text-align: center; padding: 0 4px; border-radius: 8px;
  background: #e7ecf3; color: var(--muted); font-variant-numeric: tabular-nums;
}
.xv-badge.ok { background: var(--add-bg); color: var(--add-text); }
```

```
.xv-viewbar { display: flex; gap: 4px; flex-wrap: wrap; padding: 8px 10px 0; flex: 0 0 auto; }
.xv-view {
  font: inherit; font-size: 12px; padding: 3px 12px; cursor: pointer;
  border: 1px solid var(--border); background: var(--panel); color: var(--muted); border-radius:
    5px;
}
.xv-view:hover { background: var(--fold-bg); }
.xv-view.active { background: var(--accent); border-color: var(--accent); color: #fff; }
.xv-view.empty { opacity: .55; }
.xv-count { font-variant-numeric: tabular-nums; opacity: .85; }

.xv-structural {
  display: flex; flex-wrap: wrap; gap: 12px; padding: 6px 10px; margin: 8px 10px 0;
  background: var(--fold-bg); border: 1px solid var(--border); border-radius: 5px; font-size:
    12px;
  flex: 0 0 auto;
}
.xv-st-add { color: var(--add-text); }
.xv-st-del { color: var(--del-text); }

.xv-meta {
  display: flex; align-items: center; gap: 14px;
  padding: 6px 10px 8px; color: var(--muted); font-size: 12px; flex-wrap: wrap;
  flex: 0 0 auto;
}
.xv-mode { flex: 1 1 auto; min-width: 0; }
.xv-stat { font-variant-numeric: tabular-nums; font-weight: 600; white-space: nowrap; }
.xv-stat .ok-add { color: var(--add-text); }
.xv-stat .ok-mod { color: #9a6a00; }
.xv-stat .ok-del { color: var(--del-text); }

.xv-section { margin-bottom: 14px; }
.xv-sectionhead { font-weight: 600; font-size: 12px; color: var(--muted); margin: 0 0 6px;
    padding-left: 2px; }
.xv-section.sec-add .xv-sectionhead { color: var(--add-text); }
.xv-section.sec-mod .xv-sectionhead { color: #9a6a00; }
.xv-section.sec-del .xv-sectionhead { color: var(--del-text); }

.xv-row {
  border: 1px solid var(--border); border-radius: 6px; margin-bottom: 6px;
  background: var(--panel); overflow: hidden;
}
.xv-row-add { border-left: 3px solid var(--add-strong); }
.xv-row-mod { border-left: 3px solid #ecd08a; }
.xv-row-del { border-left: 3px solid var(--del-strong); }
.xv-rowhead {
  display: flex; align-items: center; gap: 8px; padding: 4px 8px;
  background: #fafbfc; border-bottom: 1px solid var(--border); font-size: 11.5px;
}
.xv-tag { font-size: 10.5px; padding: 1px 7px; border-radius: 3px; font-weight: 600; }
.xv-tag.add { background: var(--add-bg); color: var(--add-text); }
.xv-tag.mod { background: var(--chg-bg); color: #8a6a00; }
.xv-tag.del { background: var(--del-bg); color: var(--del-text); }
.xv-ln {
  font: inherit; font-size: 11px; cursor: pointer; padding: 1px 6px;
  border: 1px solid var(--border); background: var(--panel); color: var(--accent); border-radius:
    3px;
}
.xv-ln:hover { background: #eef4ff; border-color: var(--accent); }
.xv-arrow-sep { color: var(--muted); }
.xv-changed-count { margin-left: auto; color: var(--muted); }

.xv-cells { padding: 6px 8px; display: flex; flex-wrap: wrap; gap: 4px 14px; }
.xv-cell { display: inline-flex; align-items: baseline; gap: 5px; max-width: 100%; }
```

```
.xv-k { color: var(--muted); font-size: 11.5px; flex: 0 0 auto; }
.xv-k::after { content: '： '; }
.xv-v { font-family: var(--mono); font-size: 12px; word-break: break-all; }
.xv-diff { display: inline-flex; align-items: baseline; gap: 5px; font-family: var(--mono);
    font-size: 12px; }
.xv-old { background: var(--del-bg); color: var(--del-text); padding: 0 3px; border-radius: 2px;
    text-decoration: line-through; word-break: break-all; }
.xv-arrow { color: var(--muted); font-family: inherit; }
.xv-new { background: var(--add-bg); color: var(--add-text); padding: 0 3px; border-radius: 2px;
    word-break: break-all; }
.xv-empty { color: var(--line-num); font-style: italic; font-size: 11.5px; }
/* 一行内容过多时的折叠提示（真实报表常上百列，全铺出来既看不完也难读） */
.xv-more { color: var(--muted); font-size: 11.5px; padding: 1px 6px; background: var(--fold-bg);
    border-radius: 3px; }
/* 键列：用于认出"这是哪一条记录"，给一点视觉权重（其余列是细节） */
.xv-cell-key .xv-k { color: var(--accent); font-weight: 600; }
.xv-cell-key .xv-v { font-weight: 600; }
/* 行列表是唯一可滚动区：顶部三条（sheet/视图/摘要）固定，滚动的只是它 */
.xv-rows { flex: 1 1 auto; overflow: auto; min-height: 0; padding: 0 10px 12px; }
.xv-empty-state { padding: 28px 16px; text-align: center; color: var(--muted); font-size: 13px; }
.xv-empty-state.ok { color: var(--add-text); }

/* 主区域3：Excel 表格查看器（虚拟滚动） */
.sv-host { padding: 0 !important; background: #fff !important; overflow: hidden; }
/* 该 sheet 只存在于另一侧时的占位说明（比空白面板更清楚） */
.sv-missing {
  display: flex; align-items: center; justify-content: center;
  height: 100%; min-height: 120px; padding: 24px;
  color: var(--muted); font-size: 12.5px; text-align: center;
  background: #fafbfc; border: 1px dashed var(--border); border-radius: 6px;
}
.sv-wrap { display: flex; flex-direction: column; height: 100%; min-height: 0; }
.sv-head {
  display: flex; flex: 0 0 auto; background: #f1f5fa;
  border-bottom: 1px solid var(--border); position: relative; z-index: 1;
  will-change: transform;
}
.sv-th {
  flex: 0 0 auto; padding: 4px 6px; font-size: 11.5px; font-weight: 600; color: var(--txt);
  border-right: 1px solid var(--border); white-space: nowrap; overflow: hidden;
  text-overflow: ellipsis; line-height: 1.7;
}
.sv-th-ln { width: 56px; text-align: right; color: var(--line-num); font-weight: 400; }
.sv-scroll { flex: 1 1 auto; overflow: auto; min-height: 0; overflow-anchor: none; }
.sv-body { position: relative; }
.sv-tr { display: flex; align-items: stretch; }
.sv-tr:hover { background: #f7fafd; }
.sv-td {
  flex: 0 0 auto; padding: 2px 6px; font-size: 12px; line-height: 22px;
  border-right: 1px solid #eef1f4; border-bottom: 1px solid #eef1f4;
  white-space: nowrap; overflow: hidden; text-overflow: ellipsis; font-family: var(--mono);
}
.sv-td-ln { width: 56px; text-align: right; color: var(--line-num); font-family: inherit;
    font-size: 11px; background: #fafbfc; }
.sv-add { background: var(--add-bg); }
.sv-del { background: var(--del-bg); }
.sv-mod { background: #fffdf5; }
.sv-cell-add { background: var(--add-strong) !important; }
.sv-cell-del { background: var(--del-strong) !important; }
.sv-cell-mod { background: var(--chg-bg) !important; }
.sv-hit { outline: 2px solid var(--accent); outline-offset: -2px; }

/* Excel 对比配置弹窗 */
.xc-mask {
```

```
  position: fixed; inset: 0; background: rgba(20, 28, 40, .42);
  display: flex; align-items: center; justify-content: center; z-index: 60;
}
.xc-mask[hidden] { display: none; }
.xc-modal {
  width: min(1000px, 94vw); max-height: 88vh; display: flex; flex-direction: column;
  background: var(--panel); border: 1px solid var(--border); border-radius: 10px;
  box-shadow: 0 20px 60px rgba(0, 0, 0, .32);
}
.xc-head {
  display: flex; align-items: center; gap: 10px; padding: 12px 16px;
  border-bottom: 1px solid var(--border); flex: 0 0 auto;
}
.xc-head-title { font-weight: 600; font-size: 14.5px; flex: 1 1 auto; }
.xc-body { flex: 1 1 auto; overflow: auto; padding: 12px 16px; min-height: 0; }
.xc-foot {
  display: flex; align-items: center; gap: 10px; padding: 10px 16px;
  border-top: 1px solid var(--border); flex: 0 0 auto;
}
.xc-foot-hint { color: var(--muted); font-size: 11.5px; }
.xc-foot-spacer { flex: 1 1 auto; }
.xc-foot .btn { margin: 0; }

.xc-sheetbar {
  display: flex; flex-wrap: wrap; align-items: center; gap: 8px;
  padding: 8px 10px; margin-bottom: 12px; background: var(--fold-bg);
  border: 1px solid var(--border); border-radius: 6px;
}
.xc-bar-label { font-size: 12px; color: var(--muted); }
.xc-all { font-size: 11.5px; padding: 2px 8px; }
.xc-bar-items { display: flex; flex-wrap: wrap; gap: 4px 10px; }
.xc-bar-item { font-size: 12px; display: inline-flex; align-items: center; gap: 4px; cursor:
    pointer; }
.xc-bar-item.off { opacity: .45; }

.xc-cards { display: flex; flex-direction: column; gap: 10px; }
.xc-card { border: 1px solid var(--border); border-radius: 7px; overflow: hidden; }
.xc-card.disabled { opacity: .55; }
.xc-cardhead {
  display: flex; flex-wrap: wrap; align-items: center; gap: 12px;
  padding: 7px 10px; background: #fafbfc; border-bottom: 1px solid var(--border);
}
.xc-en { display: inline-flex; align-items: center; gap: 6px; cursor: pointer; font-size: 13px; }
.xc-title { font-weight: 600; }
.xc-dim { font-size: 11.5px; color: var(--muted); font-variant-numeric: tabular-nums; }
.xc-onlyold { font-size: 11px; padding: 1px 6px; border-radius: 3px; background: var(--del-bg);
    color: var(--del-text); }
.xc-hdr { margin-left: auto; font-size: 12px; color: var(--muted); display: inline-flex;
    align-items: center; gap: 4px; }
.xc-num {
  width: 52px; font: inherit; font-size: 12px; padding: 2px 4px;
  border: 1px solid var(--border); border-radius: 4px; text-align: center;
}
.xc-to { color: var(--muted); }
.xc-cardbody { padding: 8px 10px; }
.xc-section { margin-bottom: 10px; }
.xc-section:last-child { margin-bottom: 0; }
.xc-hint { font-size: 11.5px; color: var(--muted); margin-bottom: 5px; }
.xc-keys {
  display: flex; flex-wrap: wrap; gap: 3px 12px; max-height: 120px; overflow: auto;
  border: 1px solid var(--border); border-radius: 4px; padding: 6px;
}
.xc-key { font-size: 11.5px; display: inline-flex; align-items: center; gap: 3px; cursor:
    pointer; white-space: nowrap; }
```

```
.xc-key.on { color: var(--accent); font-weight: 600; }
.xc-preview { overflow: auto; border: 1px solid var(--border); border-radius: 4px; max-height:
    170px; }
.xc-table { border-collapse: collapse; font-size: 11px; width: max-content; min-width: 100%; }
.xc-table td {
  border: 1px solid #eef1f4; padding: 2px 6px; white-space: nowrap;
  max-width: 180px; overflow: hidden; text-overflow: ellipsis;
}
.xc-rownum { color: var(--line-num); text-align: right; background: #fafbfc;
    font-variant-numeric: tabular-nums; }
.xc-tr-hd td { background: var(--fold-bg); font-weight: 600; }
.xc-tr-above td { color: var(--muted); background: #fcfcfd; }
.xc-more, .xc-more-rows { color: var(--line-num); font-size: 11px; }
.xc-more-rows { padding: 4px 6px; }
.xc-empty { color: var(--line-num); font-size: 11.5px; font-style: italic; }

// ========== 文件: tools/dsh-report.js ==========
'use strict';
/**
 * dsh-report.js — deepseek harness (dsh) 插件薄层: 一份任务 JSON 进, 一份或多份报告出。
 * 自然语言 → 任务 JSON 由 dsh 的 LLM 完成 (schema 与范例见 .dsh/skills/compare-report/SKILL.md
    ),
 * 本脚本只做机械活: 文件夹自动配对、规则化目录遍历 (walk)、分卷、逐卷子进程调用
 * tools/render.js 并聚合结果。与 render.js 的交互只走标准输出 / 退出码, 不碰其内部。
 *
 * 用法: node tools/dsh-report.js --task task.json
 * 任务 JSON (render.js 任务的超集):
 *   { title, options: { ignoreCase, ignoreEol, ignoreWhitespace,   // 缺省与页面默认一致: 全开
 *                       ignoreNewline, ignoreWidth, ignorePunct },
 *     output: "/abs/path/report.html" 或目录 (walk 模式必须是目录),
 *     maxPairsPerReport: 10,          // 可选: 每卷最多对数 (缺省 10 或环境变量
    DSH_MAX_PAIRS_PER_REPORT)
 *     pairs: [{ left, right }, …]                                  // 模式一: 显式列出每一对
 *     | source: { leftDir, rightDir,                               //
    模式二: 两文件夹按同文件名自动配对
 *                 glob: "*.docx",                                  //   可选, 默认
    *.{pdf,docx,txt,md}
 *                 list: ["a.docx", { left, right }] }              //
    可选名单: 字符串=两侧同名, 对象=显式相对/绝对路径
 *     | source: { walk: {                                          //
    模式三: 规则化目录遍历 (推荐批量场景)
 *                 root: "<文档树根>",                              //   必需: 遍历起点 (如 A)
 *                 files: "*.{pdf,docx,txt,md}",                    //   可选: 文档扩展名; html
    永远排除
 *                 left: "广东确定",                                //
    可选: 基准/原文文件名规则 (子串 | glob | re:正则);
 *                                                                  //
    缺省="广东"省份判断 (取最后一个 - 后的片段, 见下)
 *                 right: "",                                       //
    可选: 对比文件名规则; 缺省=同文件夹不含 left 的文件
 *                 volume: "folder" | <数字>                        //
    可选: 'folder'=每个文档文件夹一份报告 (缺省);
 *                                                                  //   数字=每卷最多 N
    对 (多卷加 -1/-2 后缀)
 *               } } }
 *
 * walk 模式的设计意图 (省 token / 省步骤):
 *   大模型不需要逐个列目录、也不需要枚举每一对 —— 只给 root + 规则, 代码遍历整棵树:
 *   每个"直接含文档的子文件夹" (C) 自动成为一个卷 (一份报告), 输出放到
 *   <output>/<B>/<C>.html (B = root 下第一层文件夹名; 文档直接在某 B 下时 <output>/<B>.html)。
 *   left 缺省规则 = "广东"省份版: 取文件名最后一个 '-' 后片段判断是否含"广东",
 *   避免药材名本身含"广东" (如"广东土牛膝-江西确定.pdf") 被误判为广东省份版。
 *
 * 标准输出一行结构化 JSON (日志走 stderr):
```

```
 *   { ok, reports: [{ output, stats, warnings }], stats, warnings }
 * 退出码：0 全部成功；1 任务错误（含部分对失败）；2 环境不可用（缺 render.js / 浏览器）。
 */
var fs = require('fs');
var os = require('os');
var path = require('path');
var cp = require('child_process');

var RENDER = path.join(__dirname, 'render.js');
// 每卷默认最多对数；可被任务字段 maxPairsPerReport 覆盖（walk volume:'folder' 时该上限不生效）
var MAX_PER_REPORT = parseInt(process.env.DSH_MAX_PAIRS_PER_REPORT || '10', 10);
// 并行卷数：1=串行（缺省，最稳）；>1 同时跑多个 render.js（省时但吃内存，如 DSH_PARALLEL=3）
var PARALLEL = parseInt(process.env.DSH_PARALLEL || '1', 10);

function log(s) { process.stderr.write(s + '\n'); }

// ---------- glob（* ? 与 {a,b} 展开，大小写不敏感） ----------
function expandBraces(glob) {
  var m = /\{([^{}]+)\}/.exec(glob);
  if (!m) return [glob];
  var before = glob.slice(0, m.index), after = glob.slice(m.index + m[0].length);
  var out = [];
  m[1].split(',').forEach(function (alt) {
    expandBraces(before + alt + after).forEach(function (g) { out.push(g); });
  });
  return out;
}
function globMatch(glob, name) {
  return expandBraces(glob).some(function (g) {
    var re = g.replace(/[.+^${}()|[\]\\]/g, '\\$&').replace(/\*/g, '.*').replace(/\?/g, '.');
    return new RegExp('^' + re + '$', 'i').test(name);
  });
}

// ---------- 配对 ----------
function listFiles(dir, glob) {
  return fs.readdirSync(dir).filter(function (n) {
    return fs.statSync(path.join(dir, n)).isFile() && globMatch(glob, n);
  }).sort();
}

/** 任务 → { pairs, warnings }；输入非法直接 throw（由调用方转结构化错误） */
function resolvePairs(task) {
  if (!task || typeof task !== 'object') throw new Error('任务不是合法的 JSON 对象');
  if (task.pairs && task.source) throw new Error('pairs 与 source 只能二选一');
  if (task.source && task.source.walk) throw new Error('source.walk 请用 resolveWalk（模式三）');

  if (task.source) {
    var src = task.source;
    if (!src.leftDir || !src.rightDir) throw new Error('source 需要 leftDir 与 rightDir');
    var ld = path.resolve(src.leftDir), rd = path.resolve(src.rightDir);
    if (!fs.existsSync(ld)) throw new Error('左侧文件夹不存在：' + ld);
    if (!fs.existsSync(rd)) throw new Error('右侧文件夹不存在：' + rd);
    var warnings = [], pairs = [];

    if (Array.isArray(src.list)) {
      src.list.forEach(function (item) {
        var l, r;
        if (typeof item === 'string') { l = path.join(ld, item); r = path.join(rd, item); }
        else if (item && item.left && item.right) {
          l = path.resolve(ld, item.left); r = path.resolve(rd, item.right);
        } else { warnings.push('名单条目无法识别（需为文件名或 {left,right}）：' +
    JSON.stringify(item)); return; }
        if (!fs.existsSync(l)) { warnings.push('名单文件在左侧不存在：' + l); return; }
```

```
        if (!fs.existsSync(r)) { warnings.push('名单文件在右侧不存在: ' + r); return; }
        pairs.push({ left: l, right: r });
      });
      return { pairs: pairs, warnings: warnings };
    }

    if (ld === rd) throw new Error('leftDir 与 rightDir 相同: 同目录配对请改用 pairs 或
    source.list 明确指定每一对');
    var glob = src.glob || '*.{pdf,docx,txt,md}';
    var lefts = listFiles(ld, glob);
    var rightByKey = {};   // 小写文件名 → 实际文件名
    listFiles(rd, glob).forEach(function (n) { rightByKey[n.toLowerCase()] = n; });
    var matched = {};
    lefts.forEach(function (n) {
      var r = rightByKey[n.toLowerCase()];
      if (r) { matched[r.toLowerCase()] = true; pairs.push({ left: path.join(ld, n), right:
    path.join(rd, r) }); }
      else warnings.push('仅左侧存在，未配对: ' + n);
    });
    Object.keys(rightByKey).forEach(function (k) {
      if (!matched[k]) warnings.push('仅右侧存在，未配对: ' + rightByKey[k]);
    });
    return { pairs: pairs, warnings: warnings };
  }

  if (!Array.isArray(task.pairs) || !task.pairs.length) {
    throw new Error('任务需要非空 pairs 数组，或 source 文件夹配对');
  }
  task.pairs.forEach(function (p, i) {
    if (!p || typeof p.left !== 'string' || typeof p.right !== 'string') {
      throw new Error('第 ' + (i + 1) + ' 对缺少 left/right 路径');
    }
  });
  return {
    pairs: task.pairs.map(function (p) { return { left: path.resolve(p.left), right:
    path.resolve(p.right) }; }),
    warnings: []
  };
}

// ---------- 模式三: walk 规则化目录遍历 ----------

/** 名称匹配器: 're:正则' → 正则; 含 glob 元字符 → glob; 否则子串（大小写不敏感） */
function makeMatcher(rule) {
  var src = String(rule);
  if (src.slice(0, 3) === 're:') {
    var re = new RegExp(src.slice(3), 'i');
    return function (n) { return re.test(n); };
  }
  if (/[*?{}]/.test(src)) return function (n) { return globMatch(src, n); };
  var low = src.toLowerCase();
  return function (n) { return n.toLowerCase().indexOf(low) !== -1; };
}

/** 缺省 left 规则: "广东"省份版。取文件名最后一个 '-'（或 '－'/空格）后的片段判断,
 *  避免药材名本身含"广东"（广东土牛膝-江西确定.pdf）被误判。 */
function defaultLeftMatch(name) {
  var base = String(name).replace(/\.[^.]+$/, '');
  var parts = base.split(/[-－\s]+/).filter(Boolean);
  var seg = parts.length > 1 ? parts[parts.length - 1] : base;
  return seg.indexOf('广东') !== -1;
}

/**
```

```
 * walk 模式：遍历 root 下所有子文件夹，每个"直接含匹配文档"的子文件夹成为一个卷。
 * 返回 { volumes: [{ pairs, output, title, name }], warnings }；输入非法直接 throw。
 */
function resolveWalk(task) {
  if (!task || typeof task !== 'object') throw new Error('任务不是合法的 JSON 对象');
  var w = task.source && task.source.walk;
  if (!w || typeof w !== 'object') throw new Error('source.walk 需要对象 { root, ... }');
  var root = path.resolve(w.root);
  if (!fs.existsSync(root)) throw new Error('walk 根目录不存在：' + root);
  if (!fs.statSync(root).isDirectory()) throw new Error('walk root 不是目录：' + root);

  var glob = w.files || '*.{pdf,docx,txt,md}';
  var leftMatcher = w.left ? makeMatcher(w.left) : defaultLeftMatch;
  var rightMatcher = w.right ? makeMatcher(w.right) : null;
  var outputRoot = path.resolve(task.output || 'report.html');
  var warnings = [];
  var volumes = [];

  // 遍历 root 下所有子目录（任意深度），记录 { path, name, relSegs }
  var dirs = [];
  (function walkDirs(dir, relSegs) {
    var entries;
    try { entries = fs.readdirSync(dir, { withFileTypes: true }); } catch (e) { return; }
    entries.forEach(function (en) {
      if (!en.isDirectory()) return;
      var full = path.join(dir, en.name);
      var segs = relSegs.concat(en.name);
      dirs.push({ path: full, name: en.name, segs: segs });
      walkDirs(full, segs);
    });
  })(root, []);
  dirs.sort(function (a, b) { return a.path < b.path ? -1 : a.path > b.path ? 1 : 0; });

  var docFolders = 0;
  dirs.forEach(function (d) {
    var files = listFiles(d.path, glob).filter(function (n) { return !/\.html?$/i.test(n); });
    if (!files.length) return;                          // 该文件夹没有文档 → 不是文档文件夹
    var lefts = files.filter(leftMatcher);
    var rights = files.filter(function (n) {
      if (leftMatcher(n)) return false;                 // 基准文件不做对比对象
      return rightMatcher ? rightMatcher(n) : true;
    });
    docFolders++;
    if (!lefts.length) { warnings.push('文件夹无基准文件（left 匹配不到）：' + d.path); return; }
    if (!rights.length) { warnings.push('文件夹无对比文件（right 为空）：' + d.path); return; }
    var pairs = [];
    lefts.forEach(function (l) {
      rights.forEach(function (r) {
        pairs.push({ left: path.join(d.path, l), right: path.join(d.path, r) });
      });
    });
    // 输出路径：<outputRoot>/<B>/<C>.html；B=root 下第一层，C=该文件夹名（多级用空格连接）
    var bName = d.segs[0];
    var outName = d.segs.length === 1 ? bName + '.html' : d.segs.slice(1).join(' ') + '.html';
    var output = path.join(outputRoot, bName, outName);
    // 标题：任务标题（可选）作为前缀，正文用文档文件夹名
    var title = task.title ? task.title + ' (' + d.name + ') ' : d.name;

    // 卷切分：volume:'folder'（缺省）= 整文件夹一卷；数字 = 每卷最多 N 对
    var cap = (typeof w.volume === 'number' && w.volume > 0) ? w.volume : Infinity;
    for (var i = 0; i < pairs.length; i += cap) {
      var chunk = pairs.slice(i, i + cap);
      var out = (chunk.length < pairs.length)
        ? output.replace(/\.html?$/i, function (ext) { return '-' + (i / cap + 1) + ext; })
```

```
        : output;
      // 卷内最大对数：整文件夹一卷时 = 本文件夹实际对数（让 render.js 放行，不再卡 10 对）；
      // volume 为数字时 = 该数字。
      var capForRender = (cap === Infinity) ? chunk.length : cap;
      volumes.push({ pairs: chunk, output: out, title: title, name: d.name, maxPairsPerReport:
    capForRender });
    }
  });

  if (!docFolders) throw new Error('walk: root 下没有任何包含文档的子文件夹（root=' + root +
    '）');
  return { volumes: volumes, warnings: warnings };
}

// ---------- 分卷（模式一/二） ----------
function splitVolumes(pairs, max) {
  var cap = (typeof max === 'number' && max > 0) ? max : MAX_PER_REPORT;
  var vols = [];
  for (var i = 0; i < pairs.length; i += cap) vols.push(pairs.slice(i, i + cap));
  return vols;
}
/** 单卷用原输出名；多卷在扩展名前加 -1、-2… */
function volumeOutput(output, idx, total) {
  if (total <= 1) return output;
  var ext = path.extname(output);
  return path.join(path.dirname(output), path.basename(output, ext) + '-' + (idx + 1) + ext);
}

// ---------- 调用 render.js（只走 stdout / 退出码） ----------
function runRender(subTask) {
  return new Promise(function (resolve) {
    var tmp = path.join(os.tmpdir(), 'dsh-report-' + process.pid + '-' + Date.now() + '-' +
    Math.random().toString(36).slice(2, 8) + '.json');
    fs.writeFileSync(tmp, JSON.stringify(subTask), 'utf8');
    var out = '';
    var child = cp.spawn(process.execPath, [RENDER, '--task', tmp], { stdio: ['ignore', 'pipe',
    'inherit'] });
    child.stdout.on('data', function (d) { out += d; });
    child.on('error', function (e) {
      try { fs.unlinkSync(tmp); } catch (e0) {}
      resolve({ code: 2, json: { ok: false, output: null, warnings: ['启动 render.js 失败：' +
    e.message] } });
    });
    child.on('close', function (code) {
      try { fs.unlinkSync(tmp); } catch (e0) {}
      var json = null, line = out.trim().split('\n').pop();
      try { json = JSON.parse(line); } catch (e) { /* render.js 崩溃无 JSON */ }
      resolve({ code: code, json: json });
    });
  });
}

/** 并发池跑卷：n 个同时跑，结果按输入顺序返回。 */
async function runVolumes(volumes, taskOptions, onLog) {
  var n = Math.max(1, Math.min(PARALLEL, volumes.length));
  var results = new Array(volumes.length);
  var next = 0;
  async function worker() {
    while (true) {
      var i = next++;
      if (i >= volumes.length) return;
      var v = volumes[i];
      if (onLog) onLog(i, v);
      results[i] = await runRender({ title: v.title, pairs: v.pairs, options: taskOptions,
```

```
    output: v.output,
        maxPairsPerReport: v.maxPairsPerReport });
    }
  }
  var workers = [];
  for (var k = 0; k < n; k++) workers.push(worker());
  await Promise.all(workers);
  return results;
}

// ---------- 主流程 ----------
function fail(msg, code) {
  process.stderr.write(msg + '\n');
  process.stdout.write(JSON.stringify({
    ok: false, reports: [], stats: { pairs: 0, added: 0, removed: 0, changed: 0 }, warnings: [msg]
  }) + '\n');
  process.exit(code || 1);
}

async function main() {
  if (!fs.existsSync(RENDER)) fail('找不到 tools/render.js: ' + RENDER, 2);
  var taskPath = null;
  for (var i = 2; i < process.argv.length; i++) {
    if (process.argv[i] === '--task' && i + 1 < process.argv.length) taskPath = process.argv[i +
    1];
  }
  if (!taskPath) fail('用法: node tools/dsh-report.js --task <task.json>');
  var task;
  try { task = JSON.parse(fs.readFileSync(taskPath, 'utf8')); }
  catch (e) { fail('任务文件读取/解析失败: ' + e.message); }

  var volumes = null, warnings0 = [];
  var title = task.title || '文本对比报告';
  var output = path.resolve(task.output || 'report.html');

  if (task.source && task.source.walk) {
    // 模式三: walk 直接产出卷 (每文档文件夹一卷)
    try {
      var wk = resolveWalk(task);
      volumes = wk.volumes;
      warnings0 = wk.warnings;
    } catch (e) { fail(e.message); }
  } else {
    // 模式一/二: pairs / source 文件夹配对 → 分卷
    var resolved;
    try { resolved = resolvePairs(task); }
    catch (e) { fail(e.message); }
    if (!resolved.pairs.length) {
      process.stdout.write(JSON.stringify({
        ok: false, reports: [], stats: { pairs: 0, added: 0, removed: 0, changed: 0 },
        warnings: resolved.warnings.concat(['没有可对比的文件对'])
      }) + '\n');
      process.exit(1);
    }
    var vols = splitVolumes(resolved.pairs, task.maxPairsPerReport);
    volumes = vols.map(function (v, i) {
      return {
        pairs: v,
        output: volumeOutput(output, i, vols.length),
        title: vols.length > 1 ? title + ' (' + (i + 1) + '/' + vols.length + ') ' : title
      };
    });
    warnings0 = resolved.warnings;
  }
```

```
  var pairCount = volumes.reduce(function (s, v) { return s + v.pairs.length; }, 0);
  log('共 ' + pairCount + ' 对，分 ' + volumes.length + ' 卷渲染（并行 ' + PARALLEL + '）');
  warnings0.forEach(function (w) { log('  !! ' + w); });

  var reports = [], warnings = warnings0.slice();
  var stats = { pairs: 0, added: 0, removed: 0, changed: 0 };
  var ok = true, exitCode = 0;

  var results = await runVolumes(volumes, task.options, function (idx, v) {
    log('── 卷 ' + (idx + 1) + '/' + volumes.length + '：' + v.pairs.length + ' 对 → ' +
    v.output);
  });
  var envBreak = false;
  results.forEach(function (r, i) {
    if (envBreak) return;
    if (r.code === 2 || !r.json) {   // 环境不可用或 render.js 崩溃：后续卷不必再跑
      var envMsg = r.json && r.json.warnings ? r.json.warnings.join('；')
        : 'render.js 异常退出（退出码 ' + r.code + '），未产出结构化结果';
      warnings.push('卷 ' + (i + 1) + '：' + envMsg);
      ok = false; exitCode = 2;
      envBreak = true;
      return;
    }
    reports.push({ output: r.json.output, stats: r.json.stats, warnings: r.json.warnings });
    stats.pairs += r.json.stats.pairs; stats.added += r.json.stats.added;
    stats.removed += r.json.stats.removed; stats.changed += r.json.stats.changed;
    r.json.warnings.forEach(function (w) { warnings.push('卷 ' + (i + 1) + '：' + w); });
    if (!r.json.ok) { ok = false; exitCode = 1; }
  });

  process.stdout.write(JSON.stringify({ ok: ok, reports: reports, stats: stats, warnings:
    warnings }) + '\n');
  process.exit(exitCode);
}

if (require.main === module) {
  main().catch(function (e) { fail('未预期错误：' + ((e && e.message) || e)); });
}

module.exports = {
  globMatch: globMatch, resolvePairs: resolvePairs, splitVolumes: splitVolumes,
  volumeOutput: volumeOutput, resolveWalk: resolveWalk, makeMatcher: makeMatcher
};

// ========== 文件：tools/render.js ==========
'use strict';
/**
 * render.js — 无头渲染通道：一条命令产出一份对比报告（单对 / 批量）。
 * 批量：一份报告最多 10 对（C3），超出请拆成多个任务分别调用，本工具直接报错拒绝。
 * 从 tools/spike-capture.js 起步（CDP 连接、超时、看门狗、逐页滚动收敛等基础设施直接复用），
 * 驱动真实页面复用 src/ 同一套模块（Pipeline.loadSide / compare / DocxView /
    PdfView），不重写渲染逻辑。
 *
 * 用法：node tools/render.js --task demo.json
 *   task = { title, pairs:[{left,right}], options:{ignoreCase,ignoreEol,ignoreWhitespace,
 *           ignoreNewline,ignoreWidth,ignorePunct}, output }    （options
    缺省与页面默认一致：全开）
 *
 * 标准输出（stdout）为一行结构化 JSON；日志走 stderr，不污染 stdout：
 *   { ok, output, stats:{pairs,added,removed,changed}, warnings:[] }
 * 退出码：0 成功；非 0 任务错误（1 常规失败，2 环境不可用，3 看门狗超时）。
 *
 * ── "已知的坑"逐条落实 ──
```

```
 *   1. 负/零高度 clip 会让 captureScreenshot 永久挂起 → capture() 带超时 + 调用前校验
     clipH/clipW > 2
 *   2. 协同滚动会把 scrollTop 拉回 0 → 先 SyncScroll.rebind([]) 再覆盖 rebind 为空函数
 *   3. scrollTop 要"写到稳定"（写→等→校验→再确认），末页目标值 clamp 到 scrollHeight-clientHeight
 *   4. 默认布局下面板只有约 186px 高 → 注入无头布局：隐藏主区域1/2、.pdfarea 撑满
 *   5. console 走管道会缓冲 → 日志写 stderr，并带看门狗
 *   6. 协同滚动解除后面板状态与交互态不同 → 不依赖界面滚动联动（只截面板）
 *   7. 面板可视高度有限：一次截只能拿到可见部分（长页必被截断）；而"把面板撑高再一次截整篇"
 *      已被 spike 证伪（视口外内容整片空白）→ 一页按可视高度分多次滚动切片，在 Node 内拼成
 *      整页一张图：每次切片都落在"内容确实在视口内"的已验证区间，与页高无关。
 *
 * 快照一律"按页"出图（PDF 取 .pdf-page、docx 取分页块）：只按页元素矩形与面板可视内容盒求交，
 * 不做字符级/段落级的锚点对齐——报告里的图是给人看的整页视觉记录，不需要像素级对齐文字锚点。
 */
var fs = require('fs');
var os = require('os');
var path = require('path');
var cp = require('child_process');
var zlib = require('zlib');
var Report = require(path.join(__dirname, '..', 'src', 'report.js'));

var ROOT = path.join(__dirname, '..');
var HTTP_PORT = 4310 + (process.pid % 200);
var DBG_PORT = 9410 + (process.pid % 200);

function log(s) { process.stderr.write(s + '\n'); }

// ---------- 浏览器定位：EDGE_PATH → Edge 默认路径 → Chrome 兜底，都找不到明确报错 ----------
function resolveBrowser() {
  var cands = [];
  if (process.env.EDGE_PATH) cands.push(process.env.EDGE_PATH);
  if (process.platform === 'darwin') {
    cands.push('/Applications/Microsoft Edge.app/Contents/MacOS/Microsoft Edge');
    cands.push('/Applications/Google Chrome.app/Contents/MacOS/Google Chrome');
  } else {
    cands.push('C:\\Program Files (x86)\\Microsoft\\Edge\\Application\\msedge.exe');
    cands.push('C:\\Program Files\\Microsoft\\Edge\\Application\\msedge.exe');
    cands.push('C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe');
  }
  for (var i = 0; i < cands.length; i++) {
    try { if (fs.existsSync(cands[i])) return cands[i]; } catch (e) { /* 继续 */ }
  }
  return null;
}
var BROWSER = resolveBrowser();

// ---------- 参数 ----------
function parseArgs(argv) {
  var task = null;
  for (var i = 2; i < argv.length; i++) {
    if (argv[i] === '--task' && i + 1 < argv.length) task = argv[i + 1];
  }
  return task;
}

// ---------- 基础设施（复用 spike-capture.js） ----------
function wait(ms) { return new Promise(function (r) { setTimeout(r, ms); }); }
function fetchJson(url) { return fetch(url).then(function (r) { return r.json(); }); }
var serverProc = null, edgeProc = null, ws = null, msgId = 0, pending = {};
function cdpSend(method, params, timeoutMs) {
  return new Promise(function (resolve, reject) {
    var id = ++msgId;
    var t = setTimeout(function () {
      delete pending[id];
```

```
      reject(new Error('CDP 调用超时 ' + timeoutMs + 'ms: ' + method));
    }, timeoutMs || 30000);
    pending[id] = {
      resolve: function (v) { clearTimeout(t); resolve(v); },
      reject: function (e) { clearTimeout(t); reject(e); }
    };
    ws.send(JSON.stringify({ id: id, method: method, params: params || {} }));
  });
}
function evaluate(expr) {
  return cdpSend('Runtime.evaluate', { expression: expr, awaitPromise: true, returnByValue: true
    })
    .then(function (res) {
      if (res.exceptionDetails) throw new Error('页面内执行异常: ' +
    JSON.stringify(res.exceptionDetails).slice(0, 300));
      return res.result ? res.result.value : undefined;
    });
}
function evalJson(expr) { return evaluate(expr).then(function (v) { return JSON.parse(v); }); }
function poll(expr, pred, tries, label) {
  return (function loop(n) {
    return evaluate(expr).then(function (v) {
      if (pred(v)) return v;
      if (n <= 0) throw new Error('等待超时: ' + label + ', 最后值 ' + JSON.stringify(v).slice(0,
    200));
      return wait(300).then(function () { return loop(n - 1); });
    });
  })(tries == null ? 40 : tries);
}
// 坑 1: captureScreenshot 带超时; 负/零 clip 高度会静默挂起, 调用方必须先校验 clipH/clipW
// 实测配置锁定 (dsf=2 无头 Edge): clip.scale=1 会在个别页产出确定性的横向撕裂带 (同一行位置
// 复现: 整行残影 + 贯穿整页的横线, 重试无用); fromSurface=false 直接截出全白图。
// clip.scale=2 + captureBeyondViewport=true + fromSurface=true 是 spike 以来一直验证干净的路径,
// 多出的 2x 像素在 Node 内 box 降采样回 2x (见 capturePage), 细节无损、体积可控。
// 注意: 个别文档页在某种 clip.scale 下会踩到 Chromium 确定性的光栅化 bug (同一位置、同一形态
// 的噪点带, 重截多少次都一模一样) ——与随机撕裂不同, 重试无解, 必须换一个 clip.scale 再截
//  (光栅化路径改变即避开), capturePage 的坏帧兜底链实现了这一降级。
function capture(clip, scale) {
  return cdpSend('Page.captureScreenshot', {
    format: 'png',
    clip: { x: clip.x, y: clip.y, width: clip.width, height: clip.height, scale: scale || 1 },
    captureBeyondViewport: true,
    fromSurface: true
  }, 60000).then(function (r) { return Buffer.from(r.data, 'base64'); });
}
// ---------- PNG 编解码 (拼图用): 切片各自成图, 必须能在 Node 内合成整页一张图 ----------
function paeth(a, b, c) {
  var p = a + b - c, pa = Math.abs(p - a), pb = Math.abs(p - b), pc = Math.abs(p - c);
  return pa <= pb && pa <= pc ? a : pb <= pc ? b : c;
}

/** 解码 8bit / 非隔行的 RGB(2) 或 RGBA(6) PNG (Chrome 截图即此二者) → { w, h, rgba };
 *  其它格式返回 null (调用方据此判定"截到坏图")。逐行 unfilter, Sub/Up/Average/Paeth 全支持。 */
function decodePng(buf) {
  if (buf.length < 33 || buf.readUInt32BE(0) !== 0x89504e47) return null;
  var w = buf.readUInt32BE(16), h = buf.readUInt32BE(20);
  var depth = buf[24], ctype = buf[25], interlace = buf[28];
  if (depth !== 8 || interlace !== 0 || (ctype !== 2 && ctype !== 6)) return null;
  if (w < 1 || h < 1 || w > 20000 || h > 20000) return null;
  var bpp = ctype === 6 ? 4 : 3, stride = w * bpp;
  var off = 8, idat = [];
  while (off + 8 <= buf.length) {
    var len = buf.readUInt32BE(off), type = buf.toString('ascii', off + 4, off + 8);
    if (type === 'IDAT') idat.push(buf.slice(off + 8, off + 8 + len));
```

```
    off += 12 + len;
    if (type === 'IEND') break;
  }
  var raw;
  try { raw = zlib.inflateSync(Buffer.concat(idat)); } catch (e) { return null; }
  if (raw.length < (stride + 1) * h) return null;
  var rgba = Buffer.alloc(w * h * 4);
  var prev = Buffer.alloc(stride), cur = Buffer.alloc(stride), pos = 0, x, y;
  for (y = 0; y < h; y++) {
    var f = raw[pos++];
    raw.copy(cur, 0, pos, pos + stride); pos += stride;
    if (f === 1) for (var i = bpp; i < stride; i++) cur[i] = (cur[i] + cur[i - bpp]) & 255;
    else if (f === 2) for (var j = 0; j < stride; j++) cur[j] = (cur[j] + prev[j]) & 255;
    else if (f === 3) for (var k = 0; k < stride; k++) cur[k] = (cur[k] + ((k < bpp ? 0 : cur[k -
    bpp]) + prev[k]) >> 1) & 255;
    else if (f === 4) for (var m = 0; m < stride; m++) cur[m] = (cur[m] + paeth(m < bpp ? 0 :
    cur[m - bpp], prev[m], m < bpp ? 0 : prev[m - bpp])) & 255;
    for (x = 0; x < w; x++) {
      var s = x * bpp, d = (y * w + x) * 4;
      rgba[d] = cur[s]; rgba[d + 1] = cur[s + 1]; rgba[d + 2] = cur[s + 2];
      rgba[d + 3] = ctype === 6 ? cur[s + 3] : 255;
    }
    var t = prev; prev = cur; cur = t;
  }
  return { w: w, h: h, rgba: rgba };
}

var CRC_TABLE = (function () {
  var t = new Int32Array(256);
  for (var n = 0; n < 256; n++) {
    var c = n;
    for (var k = 0; k < 8; k++) c = (c & 1) ? (0xedb88320 ^ (c >>> 1)) : (c >>> 1);
    t[n] = c;
  }
  return t;
})();
function crc32(buf) {
  var c = -1;
  for (var i = 0; i < buf.length; i++) c = CRC_TABLE[(c ^ buf[i]) & 255] ^ (c >>> 8);
  return (c ^ -1) >>> 0;
}
function pngChunk(type, data) {
  var head = Buffer.alloc(8), crc = Buffer.alloc(4);
  head.writeUInt32BE(data.length, 0);
  head.write(type, 4, 'ascii');
  crc.writeUInt32BE(crc32(Buffer.concat([head.slice(4), data])), 0);
  return Buffer.concat([head, data, crc]);
}
/** 编码 RGBA → PNG（8bit RGBA、过滤器固定 0，体积交给 zlib） */
function encodePng(w, h, rgba) {
  var stride = w * 4, raw = Buffer.alloc((stride + 1) * h);
  for (var y = 0; y < h; y++) {
    raw[y * (stride + 1)] = 0;
    rgba.copy(raw, y * (stride + 1) + 1, y * stride, (y + 1) * stride);
  }
  var ihdr = Buffer.alloc(13);
  ihdr.writeUInt32BE(w, 0); ihdr.writeUInt32BE(h, 4);
  ihdr[8] = 8; ihdr[9] = 6; ihdr[10] = 0; ihdr[11] = 0; ihdr[12] = 0;
  return Buffer.concat([
    Buffer.from([0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a, 0x1a, 0x0a]),
    pngChunk('IHDR', ihdr),
    pngChunk('IDAT', zlib.deflateSync(raw, { level: 6 })),
    pngChunk('IEND', Buffer.alloc(0))
  ]);
```

```
}

/** 整数倍 box 降采样（f=2：2×2 块取均）→ 新对象 {w,h,rgba}；f<=1 原样返回 */
function downscale(img, f) {
  if (!(f > 1)) return img;
  var w = Math.floor(img.w / f), h = Math.floor(img.h / f);
  var out = Buffer.alloc(w * h * 4);
  for (var y = 0; y < h; y++) {
    for (var x = 0; x < w; x++) {
      var r = 0, g = 0, b = 0, a = 0;
      for (var dy = 0; dy < f; dy++) {
        for (var dx = 0; dx < f; dx++) {
          var i = ((y * f + dy) * img.w + (x * f + dx)) * 4;
          r += img.rgba[i]; g += img.rgba[i + 1]; b += img.rgba[i + 2]; a += img.rgba[i + 3];
        }
      }
      var n = f * f, d = (y * w + x) * 4;
      out[d] = Math.round(r / n); out[d + 1] = Math.round(g / n);
      out[d + 2] = Math.round(b / n); out[d + 3] = Math.round(a / n);
    }
  }
  return { w: w, h: h, rgba: out };
}

/** 把切片的 [srcTop, h) 行贴到整页画布的 dstRow（宽度取两者较小值，防越界）；返回写入行数 */
function blit(dst, dstW, src, srcTop, dstRow) {
  var dstH = Math.floor(dst.length / 4 / dstW);
  var rows = Math.max(0, Math.min(src.h - srcTop, dstH - dstRow));
  var cols = Math.min(src.w, dstW);
  for (var y = 0; y < rows; y++) {
    src.rgba.copy(dst, (dstRow + y) * dstW * 4, ((srcTop + y) * src.w) * 4, ((srcTop + y) * src.w
    + cols) * 4);
  }
  return rows;
}

/** 暗像素占比（%）：抽样步长 32px，用于识别空白图 / 坏帧 */
function darkRatio(rgba, w, h) {
  var dark = 0, tot = 0;
  for (var y = 0; y < h; y += 8) {
    for (var x = 0; x < w; x += 32) {
      var i = (y * w + x) * 4;
      if ((rgba[i] + rgba[i + 1] + rgba[i + 2]) / 3 < 128) dark++;
      tot++;
    }
  }
  return tot ? +(100 * dark / tot).toFixed(2) : 0;
}

/** 高饱和像素占比（%）：识别"五颜六色花纹"坏帧。文档页近灰阶（差异标注是浅红/浅绿，
 *  max(r,g,b)-min(r,g,b) 一般 < 60），而 Chromium 偶发返回的撕裂坏帧是满饱和随机噪点
 *  （差值动辄 200+，占比常超 50%）。阈值取 80，天然避开浅色标注不误判。 */
function colorfulRatio(rgba, w, h) {
  var col = 0, tot = 0;
  for (var y = 0; y < h; y += 8) {
    for (var x = 0; x < w; x += 32) {
      var i = (y * w + x) * 4;
      var mx = Math.max(rgba[i], rgba[i + 1], rgba[i + 2]);
      var mn = Math.min(rgba[i], rgba[i + 1], rgba[i + 2]);
      if (mx - mn > 80) col++;
      tot++;
    }
  }
  return tot ? +(100 * col / tot).toFixed(2) : 0;
```

```
}

/** 噪点行带扫描：把整页纵向等分成 20 条，逐条统计高饱和像素占比，返回最大一条的占比（%）。
 *  整页均值检测的盲区：撕裂坏帧常以"局部横带"出现（一条 5% 页高的噪点带只占全图 5% 高饱和，
 *  均值 5% 就漏检）；带扫描抓局部峰值，一条带里噪点超 40% 即判坏帧。真彩色图表页里纯色块
 *  （红字、绿章）一般横贯整页、不会只挤在某一条窄带里且达到噪点密度，不易误判。 */
function colorfulBandRatio(rgba, w, h) {
  var BANDS = 20, worst = 0;
  for (var b = 0; b < BANDS; b++) {
    var y0 = Math.floor(h * b / BANDS), y1 = Math.max(y0 + 1, Math.floor(h * (b + 1) / BANDS));
    var col = 0, tot = 0;
    for (var y = y0; y < y1; y += 4) {
      for (var x = 0; x < w; x += 16) {
        var i = (y * w + x) * 4;
        var mx = Math.max(rgba[i], rgba[i + 1], rgba[i + 2]);
        var mn = Math.min(rgba[i], rgba[i + 1], rgba[i + 2]);
        if (mx - mn > 80) col++;
        tot++;
      }
    }
    var r = tot ? 100 * col / tot : 0;
    if (r > worst) worst = r;
  }
  return +worst.toFixed(2);
}
function cleanup() {
  clearTimeout(WATCHDOG);
  try { if (ws) ws.close(); } catch (e) {}
  try {
    if (edgeProc) {
      if (process.platform === 'win32') cp.execSync('taskkill /PID ' + edgeProc.pid + ' /T /F
    2>nul');
      else { try { process.kill(-edgeProc.pid, 'SIGKILL'); } catch (e0) {} edgeProc.kill(); }
    }
  } catch (e2) {}
  try { if (serverProc) serverProc.kill(); } catch (e3) {}
}
// 坑 5：看门狗，卡住时给出失败 JSON 而非静默挂死。
// 原实现：单次 240s 定时器全程不重置 → 一批超过 10 对（含 OCR 扫描件）时会误杀。
// 改为 armWatchdog()：每对开始前重挂，看门狗只负责"当前这一对"不卡死，全程时长不受 240s 限制。
var WATCHDOG = null;
var WATCHDOG_MS = parseInt(process.env.DSH_WATCHDOG_MS || '600000', 10);
function armWatchdog() {
  if (WATCHDOG) clearTimeout(WATCHDOG);
  WATCHDOG = setTimeout(function () {
    process.stderr.write('!! 看门狗触发：' + (WATCHDOG_MS / 1000) + 's 当前对未完成，强制退出\n');
    process.stdout.write(JSON.stringify({ ok: false, output: null, stats: { pairs: 0, added: 0,
    removed: 0, changed: 0 }, warnings: ['看门狗超时'] }) + '\n');
    try { process.exit(3); } catch (e) {}
  }, WATCHDOG_MS);
}
armWatchdog();

// ---------- 逐页快照采集（B4） ----------
/** 坑 3：把面板滚到指定位置并确认"稳住了"；目标值在页面内 clamp 到 scrollHeight-clientHeight
 *  （末页目标会超过滚动上限，不 clamp 会永远判定"没收敛"，白白重试多轮） */
function scrollPanelTo(pid, target) {
  var want = -1;
  return (async function () {
    for (var attempt = 1; attempt <= 6; attempt++) {
      want = await evaluate('(function(){var p=document.getElementById(' + JSON.stringify(pid) +
    ');' +
        'var want=Math.min(Math.max(0,' + Math.round(target) +
    '),p.scrollHeight-p.clientHeight);' +
```

```
        'p.scrollTop=want;return Math.round(want);})()');
      await wait(220);
      var now = await evaluate('document.getElementById(' + JSON.stringify(pid) + ').scrollTop');
      if (Math.abs(now - want) < 2) {
        await wait(180);   // 再观察一次：确认没被后续帧拉回
        var again = await evaluate('document.getElementById(' + JSON.stringify(pid) +
    ').scrollTop');
        if (Math.abs(again - want) < 2) return { ok: true, target: want, attempts: attempt };
      }
    }
    return { ok: false, target: want, attempts: 6 };
  })();
}

/**
 * 切片几何：页元素矩形 ∩ 面板可视内容盒，全部视口坐标。
 * 可视内容盒用 clientWidth/clientHeight 推（二者已排除滚动条）：面板是 overflow:auto，
 * 若用 pr.right-border-padding 去算，会把滚动条那一条也算成可视内容，截图右/下边缘
 * 会带上一条面板底色 + 相邻面板的边线（旧实现"左侧截图把右侧也截进来"的来源之一）。
 *   - 纵向求交：一次只截"确实看得见"的那段，长页靠多次滚动切片覆盖
 *   - 横向求交：页比面板宽时（docx 缩放不足，元素矩形会伸出面板）按元素矩形取
 *     会把相邻面板的内容一起截进来，必须夹在可视内容盒内
 *   - offStart/offEnd：该切片对应的"页内像素"纵向范围，供拼回整页
 *   - base：页顶在滚动内容中的坐标（扣掉面板自身 border+padding），滚动定位用
 */
function sliceGeom(pid, sel, k) {
  return evalJson('JSON.stringify((function(){var p=document.getElementById(' +
    JSON.stringify(pid) + ');' +
    'var el=p.querySelectorAll(' + JSON.stringify(sel) + ')[' + k + '];' +
    'var er=el.getBoundingClientRect(),pr=p.getBoundingClientRect();' +
    'var cs=getComputedStyle(p),q=function(n){return parseFloat(cs[n])||0;};' +
    'var bt=q("borderTopWidth"),bl=q("borderLeftWidth"),pt=q("paddingTop"),pl=q("paddingLeft");' +
    'var vl=pr.left+bl+pl,vr=vl+p.clientWidth-pl-q("paddingRight");' +
    'var vt=pr.top+bt+pt,vb=vt+p.clientHeight-pt-q("paddingBottom");' +
    'var x0=Math.max(er.left,vl),x1=Math.min(er.right,vr);' +
    'var y0=Math.max(er.top,vt),y1=Math.min(er.bottom,vb);' +
    'return {clipX:x0+window.scrollX,clipY:y0+window.scrollY,clipW:x1-x0,clipH:y1-y0,' +
    'pageH:er.height,pageW:er.width,offStart:y0-er.top,offEnd:y1-er.top,' +
    'base:er.top-pr.top+p.scrollTop-(bt+pt),visH:vb-vt,visW:vr-vl};})())');
}

/**
 * 按页出图：一页 = 多次"滚动步进 + 截可视区"，在 Node 内拼成整页一张图。
 * 面板可视高度有限，一次截只能拿到可见部分（长页必被截断）；把面板撑高再一次截整篇又被
 * spike 证伪（视口外内容整片空白）。切片拼图两难皆避：每片都在已验证的"内容确实可见"区间。
 * 返回 { rgba, w, h, filled, dark }（仍未编码，交给调用方校验后再编码）。
 */
async function capturePage(pid, sel, k, scale, label) {
  var g0 = await sliceGeom(pid, sel, k);
  if (!(g0.pageH > 2) || !(g0.clipW > 2)) {
    throw new Error(label + ' 第 ' + (k + 1) + ' 页尺寸异常（页高 ' + g0.pageH.toFixed(1) +
      '，可视宽 ' + g0.clipW.toFixed(1) + '）');
  }
  var canvas = null, W = 0, H = 0, ratio = 0, filled = 0, slices = 0;
  var off = 0, guard = Math.ceil(g0.pageH / Math.max(40, g0.visH)) + 8;
  while (off < g0.pageH - 1 && guard-- > 0) {
    var sc = await scrollPanelTo(pid, g0.base + off);
    var g = await sliceGeom(pid, sel, k);
    // 坑 1：负/零高度 clip 会永久挂起，先校验
    if (!(g.clipH > 2) || !(g.clipW > 2)) {
      throw new Error(label + ' 第 ' + (k + 1) + ' 页 ' + Math.round(off) + 'px
    处不在视口内（clipH=' +
        g.clipH.toFixed(1) + ', scrollTop 目标 ' + sc.target + ', 稳定=' + sc.ok + '）');
    }
```

```
    // 切片级坏帧兜底：captureScreenshot 偶发返回撕裂坏帧（彩色噪点，常以局部横带出现），
    // 拼进整页就是"五颜六色花纹"。坏帧分两类，处置链逐级升级：
    //   随机撕裂：等一帧重截即恢复 → 前 2 次同 scale 重截
    //   确定性光栅化 bug：某一 clip.scale 下同一位置必现同一噪点带（重截无用）→ 换 scale 再截
    // 真彩色图表页各档都超限则原样采用并记日志，不会死循环。
    var img = null, sliceCol = 0, sliceBand = 0;
    // 档位：[重截次数, clip.scale 倍率]——1x=原 scale，1.5x/0.75x 改变光栅化路径避开确定性坏帧
    var ladder = [[2, 1], [2, 1.5], [2, 0.75]];
    var badFrame = false;
    outer:
    for (var rung = 0; rung < ladder.length; rung++) {
      var tries = ladder[rung][0], sc2 = scale * ladder[rung][1];
      for (var sAttempt = 1; sAttempt <= tries; sAttempt++) {
        var buf = await capture({ x: g.clipX, y: g.clipY, width: g.clipW, height: g.clipH }, sc2);
        img = decodePng(buf);
        if (!img) throw new Error(label + ' 第 ' + (k + 1) + ' 页切片解码失败（非 8bit RGB/RGBA
    PNG）');
        // 截图像素比 = dpr × clip.scale（dpr=2 强制 + scale=2 → 4x）。目标成图固定
    2x（与文档约定的
        // 1224×1584/页一致）：高于 2x 的部分 box 降采样回来，细节不变（画布本来就是 2x 光栅化）、
        // 体积只有 4x 直存的 1/4 左右。
        img = downscale(img, Math.max(1, Math.round(img.w / g.clipW / 2)));
        sliceCol = colorfulRatio(img.rgba, img.w, img.h);
        sliceBand = colorfulBandRatio(img.rgba, img.w, img.h);
        if (sliceCol <= 3 && sliceBand <= 40) {
          if (rung > 0) log('      ' + label + ' 第 ' + (k + 1) + ' 页切片换 clip.scale=' +
            ladder[rung][1] + 'x 后恢复干净（均值 ' + sliceCol + '%，带峰 ' + sliceBand + '%）');
          break outer;
        }
        badFrame = true;
        log('      ' + label + ' 第 ' + (k + 1) + ' 页切片疑似坏帧（高饱和均值 ' + sliceCol +
          '%，行带峰值 ' + sliceBand + '%，scale 档 ' + ladder[rung][1] + 'x），第 ' +
          (rung * 2 + sAttempt) + ' 次重截');
        await wait(300);
        await scrollPanelTo(pid, g0.base + off);   // 重新稳定滚动位，给合成器一帧新内容
      }
    }
    if (badFrame && (sliceCol > 3 || sliceBand > 40)) {
      log('      !! ' + label + ' 第 ' + (k + 1) + ' 页切片换档重截后仍疑似坏帧（均值 ' +
    sliceCol +
        '%，带峰 ' + sliceBand + '%），按原样采用');
    }
    if (!canvas) {
      // 像素比由实拍图反推（dpr × clip.scale ÷ 降采样倍率），不假设 devicePixelRatio 的具体值
      ratio = img.w / g.clipW;
      W = img.w;
      H = Math.max(1, Math.round(g.pageH * ratio));
      canvas = Buffer.alloc(W * H * 4, 255);   // 白底：任何未覆盖的行都是页外空白
    }
    // 滚动到底被 clamp 时，末尾切片会与上一片重叠：必须按"已覆盖到的页内位置"裁掉重叠部分，
    // 否则同一段内容会被当成新内容往后贴（表现为页面内容重复出现一截）
    var coveredCss = filled / ratio;
    var skipCss = Math.max(0, coveredCss - g.offStart);
    var srcTop = Math.round(skipCss * ratio);
    var dstRow = Math.max(filled, Math.min(H - 1, Math.round((g.offStart + skipCss) * ratio)));
    if (img.h - srcTop > 0) filled = Math.min(H, dstRow + blit(canvas, W, img, srcTop, dstRow));
    slices++;
    off = g.offEnd;   // 推进到"本片已覆盖到的页内位置"，被 clamp 时也不会原地打转
  }
  return { rgba: canvas, w: W, h: H, filled: filled, slices: slices,
    dark: darkRatio(canvas, W, H), colorful: colorfulRatio(canvas, W, H),
    band: colorfulBandRatio(canvas, W, H) };
}
```

```
/**
 * 逐页快照：返回 [{data,w,h}]；任何一页失败即抛错（该对计入 warning）。
 * PDF 优先走"直接光栅化"通道：页面内离屏 canvas 由 pdf.js 重绘后 toDataURL 读像素，
 * 完全绕开 Chromium 合成器截图（确定性噪点带 bug 的根源）；通道不可用再回退截图链。
 */
async function snapSide(side, pid, sel, label, directPdf) {
  var n = await evaluate('document.getElementById(' + JSON.stringify(pid) + ').querySelectorAll('
    + JSON.stringify(sel) + ').length');
  var pages = [];

  // ── 通道一（仅 PDF）：直接光栅化。与 DOM/滚动/合成器无关，所见即 PDF 真实内容 ──
  if (directPdf) {
    var okAll = true;
    for (var k0 = 0; k0 < n; k0++) {
      // 等 DOM 侧该页渲染完成（文档已加载、页对象可用；不依赖其画布，只需页数/几何稳定）
      await poll('JSON.stringify({ok:(function(){var r=' +
        'PdfView.isPageRendered(' + JSON.stringify(side) + ',' + (k0 + 1) + ');' +
        'return r;})()})',
        function (v) { return JSON.parse(v).ok; }, 120, label + ' 第 ' + (k0 + 1) + '
    页渲染完成');
      var r = await evaluate('PdfView.rasterizePage(' + JSON.stringify(side) + ',' + (k0 + 1) +
    ',1,false)');
      if (!r || !r.data) { okAll = false; log('      !! ' + label + ' 第 ' + (k0 + 1) + '
    页直接光栅化失败，回退截图'); break; }
      // 像素完整性校验：PDF 页不会是纯白（dark≈0 说明渲染失败，不能收）
      var raw = Buffer.from(r.data.split(',')[1], 'base64');
      var probe = decodePng(raw);
      var dk = probe ? darkRatio(probe.rgba, probe.w, probe.h) : 0;
      var col = probe ? colorfulRatio(probe.rgba, probe.w, probe.h) : 100;
      if (!probe || !(dk > 0.05) || col > 20) {
        okAll = false;
        log('      !! ' + label + ' 第 ' + (k0 + 1) + ' 页直接光栅化结果异常（dark=' + dk +
          '%，高饱和=' + col + '%），回退截图');
        break;
      }
      pages.push({ data: r.data, w: r.w, h: r.h });
      log('      ' + label + ' 第 ' + (k0 + 1) + '/' + n + ' 页：直接光栅化 ' + r.w + '×' + r.h +
        ' (dark=' + dk + '%), ' + (raw.length / 1024).toFixed(0) + 'KB');
    }
    if (okAll) return pages;
    pages = [];   // 回退整侧重截，不混用两条通道的几何
  }

  // ── 通道二：面板滚动 + 合成器截图（含换档降级链） ──
  var wide = await evalJson('JSON.stringify((function(){var p=document.getElementById(' +
    JSON.stringify(pid) + ');' +
    'var el=p.querySelectorAll(' + JSON.stringify(sel) + ')[0];if(!el)return {pageW:0,visW:0};' +
    'var cs=getComputedStyle(p),pr=p.getBoundingClientRect();' +
    'return {pageW:el.getBoundingClientRect().width,' +
    'visW:pr.width-(parseFloat(cs.borderLeftWidth)||0)-(parseFloat(cs.borderRightWidth)||0)' +
    '-(parseFloat(cs.paddingLeft)||0)-(parseFloat(cs.paddingRight)||0)};})())');
  if (wide.pageW > wide.visW + 1) {
    log('      ' + label + ' 页宽 ' + Math.round(wide.pageW) + 'px 超出面板可视宽 ' +
      Math.round(wide.visW) + 'px，横向会按可视区裁边（不越界截到另一侧面板）');
  }
  for (var k = 0; k < n; k++) {
    // 坑 6：pdf.js 的 page.render() 是异步的，canvas 尺寸一旦分配 loadedExpr 就返回 true，
    // 但画布可能还没画完（字体/CMap 解析慢或渲染任务排队）。逐页轮询"该页已渲染完成"标记。
    if (typeof PdfView !== 'undefined' && PdfView.isPageRendered) {
      await poll('JSON.stringify({ok:(function(){var r=' +
        'PdfView.isPageRendered(' + JSON.stringify(side) + ',' + (k + 1) + ');' +
        'return r;})()})',
        function (v) { return JSON.parse(v).ok; }, 120, label + ' 第 ' + (k + 1) + ' 页渲染完成');
    }
```

```
    await wait(120);
    // 坑 7：滚动+截图组合下 CDP capture 可能拿到几何未稳定/合成未更新的坏帧（随机空白、
    // 偶发 clip 高度错误、局部噪点横带）。从"结果端"兜底：校验拼图是否覆盖整页、是否有内容、
    // 是否有坏帧残带（整页均值与行带峰值双重判定），异常重来（最多 3 次），不依赖具体机制。
    var shot = null;
    for (var attempt = 1; attempt <= 3; attempt++) {
      shot = await capturePage(pid, sel, k, 2, label);
      var covered = shot.filled >= shot.h * 0.98;
      if (shot.dark > 0.15 && covered && shot.colorful <= 3 && shot.band <= 40) break;
      log('      ' + label + ' 第 ' + (k + 1) + '/' + n + ' 页截图异常（dark=' + shot.dark +
    '%, ' +
        '高饱和均值=' + shot.colorful + '%, 行带峰值=' + shot.band + '%, 覆盖 ' + shot.filled +
    '/' +
        shot.h + 'px），第 ' + attempt + ' 次重试');
      await wait(300);
    }
    var buf = encodePng(shot.w, shot.h, shot.rgba);
    pages.push({ data: '' + buf.toString('base64'), w: shot.w, h: shot.h });
    log('      ' + label + ' 第 ' + (k + 1) + '/' + n + ' 页：' + shot.slices + ' 片拼成 ' +
      shot.w + '×' + shot.h + '（覆盖 ' + shot.filled + '/' + shot.h + 'px, dark=' + shot.dark +
    '%）, ' +
      (buf.length / 1024).toFixed(0) + 'KB');
  }
  return pages;
}

// ---------- 单对处理 ----------
/** 把 M（{optIgnoreCase:0|1,…}）转成 Compute 的 options
    原始文本视图强制按行对比（ignoreNewline=false） */
function rawOptions(M) {
  var map = { optIgnoreCase: 'ignoreCase', optIgnoreEol: 'ignoreEol', optIgnoreWhitespace:
    'ignoreWhitespace',
    optIgnoreWidth: 'ignoreWidth', optIgnorePunct: 'ignorePunct' };
  var o = { ignoreNewline: false };
  for (var id in map) o[map[id]] = !!M[id];
  return o;
}
/** 页面内驱动：加载两侧 → 比较 → 取结果 → 逐页快照。返回 {leftName,rightName,result,shots} */
async function renderPair(pair, opts, idx, warnings) {
  var left = path.resolve(pair.left), right = path.resolve(pair.right);
  if (!fs.existsSync(left)) throw new Error('左侧文件不存在：' + left);
  if (!fs.existsSync(right)) throw new Error('右侧文件不存在：' + right);
  var kindL = /\.docx$/i.test(left) ? 'docx' : /\.pdf$/i.test(left) ? 'pdf' : 'none';
  var kindR = /\.docx$/i.test(right) ? 'docx' : /\.pdf$/i.test(right) ? 'pdf' : 'none';
  log('  [对 ' + (idx + 1) + '] ' + path.basename(left) + ' ↔ ' + path.basename(right) + '（L=' +
    kindL + ' R=' + kindR + '）');

  // 选项先设好（页面自动对比会按当前选项跑），再加载。缺省与页面默认一致（全开）
  var M = { optIgnoreCase: 1, optIgnoreEol: 1, optIgnoreWhitespace: 1, optIgnoreNewline: 1,
    optIgnoreWidth: 1, optIgnorePunct: 1 };
  if (opts) {
    if ('ignoreCase' in opts) M.optIgnoreCase = opts.ignoreCase ? 1 : 0;
    if ('ignoreEol' in opts) M.optIgnoreEol = opts.ignoreEol ? 1 : 0;
    if ('ignoreWhitespace' in opts) M.optIgnoreWhitespace = opts.ignoreWhitespace ? 1 : 0;
    if ('ignoreNewline' in opts) M.optIgnoreNewline = opts.ignoreNewline ? 1 : 0;
    if ('ignoreWidth' in opts) M.optIgnoreWidth = opts.ignoreWidth ? 1 : 0;
    if ('ignorePunct' in opts) M.optIgnorePunct = opts.ignorePunct ? 1 : 0;
  }
  await evaluate('(function(){var M=' + JSON.stringify(M) + ';for(var id in M)' +
    'document.getElementById(id).checked=!!M[id];return 1;})()');
  // 扫描版 PDF 自动识别：无头模式不弹确认框，检测到无文字层即自动跑 OCR（见 pipeline.js）
  await evaluate('window.__HEADLESS_OCR__ = true; 1');
  await evaluate('Pipeline.loadSide("L", ' + JSON.stringify(left.replace(/\\/g, '/')) + ')');
  await evaluate('Pipeline.loadSide("R", ' + JSON.stringify(right.replace(/\\/g, '/')) + ')');
```

```
// 等两侧渲染完成：PDF 等画布、docx 等 renderAsync + 文本提取
function loadedExpr(side, kind) {
  var pid = side === 'L' ? 'pdfLeft' : 'pdfRight';
  if (kind === 'pdf') {
    return 'JSON.stringify({ok:(function(){var p=document.getElementById(' +
  JSON.stringify(pid) + ');' +
      'var c=p?p.querySelectorAll(".pdf-page canvas"):[];' +
      'return c.length>0&&Array.prototype.every.call(c,function(x){return
  x.width>0&&x.height>0;});})()})';
  }
  if (kind === 'docx') {
    return 'JSON.stringify({ok:(function(){var d=typeof
  DocxView!=="undefined"?DocxView._debug("' + side + '"):null;' +
      'return !!(d&&d.loaded&&d.hasText);})()})';
  }
  // 纯文本：无面板，等编辑区写入文本即可（.CodeMirror 按 DOM 顺序 = L、R）
  return 'JSON.stringify({ok:(function(){var eds=document.querySelectorAll(".CodeMirror");' +
    'return eds.length>1&&eds[' + (side === 'L' ? 0 : 1) +
  '].CodeMirror.getValue().length>0;})()})';
}
await poll(loadedExpr('L', kindL), function (v) { return JSON.parse(v).ok; }, 60, 'L 面板渲染');
await poll(loadedExpr('R', kindR), function (v) { return JSON.parse(v).ok; }, 60, 'R 面板渲染');
await evaluate('document.fonts.ready.then(function(){return 1})');

// 扫描版 PDF：等两侧 OCR 落定（自动识别在后台跑）。失败记 warning，报告仍继续（该侧文本为空）。
if (kindL === 'pdf' || kindR === 'pdf') {
  var ocrSides = [];
  if (kindL === 'pdf') ocrSides.push('L');
  if (kindR === 'pdf') ocrSides.push('R');
  for (var oi = 0; oi < ocrSides.length; oi++) {
    var os = ocrSides[oi];
    var st = await poll('JSON.stringify({s:(typeof Pipeline.getOcrState==="function")?' +
      'Pipeline.getOcrState(' + JSON.stringify(os) + '):"none",ed:(function(){var
  eds=document.querySelectorAll(".CodeMirror");' +
      'return eds.length>1?eds[' + (os === 'L' ? 0 : 1) +
  '].CodeMirror.getValue().length:0;})()})',
      function (v) {
        var s = JSON.parse(v);
        // 文字层 PDF 的 OCR 状态恒为 idle → 直接通过；
        // 扫描版 PDF：pending/running = OCR 或缓存注入仍在途，必须等落定（done/failed）——
        // 绝不能因编辑器残留上一对的文本（s.ed > 0）短路，否则 textItems 尚未注入，
        // 报告快照的差异标注会画不出来（历史 bug：多对连跑时第 2 对起的扫描页无标注）。
        if (s.s === 'failed') return true;                      // OCR
  失败：按原设计继续（该侧无文本，进 warnings）
        if (s.s === 'pending' || s.s === 'running') return false;
        return s.s === 'idle' || s.s === 'done';                // idle（文字层）或
  done（OCR/缓存注入完成）
      }, 600, os + ' 侧 OCR 完成');   // 600×300ms≈180s，覆盖首次模型加载 + 大扫描件逐页识别
    var stj = JSON.parse(st);
    if (stj.s === 'failed') {
      var msg = '第 ' + (idx + 1) + ' 对 ' + os + ' 侧扫描版 OCR 失败（报告该侧无文本）';
      log('  !! ' + msg);
      warnings.push(msg);
    } else if (stj.s === 'pending' || stj.s === 'running') {
      log('  ' + os + ' 侧是扫描版 PDF，已自动 OCR（' + stj.s + ' → 完成）');
    }
  }
}

// 报告默认执行一遍"修整"（同界面"修整"按钮：Norm.tidyText 去全部空白整理为单行）
  缩小报告空间占用。
// 先等编辑器写入面板原文——PDF 提词晚于画布渲染，否则修整会被随后到来的原文覆盖。
// 面板标注不受影响：编辑器文本与面板原文不一致时，Pipeline.syncPanelTextAnnotations
```

```
  按面板原文另算标注。
await poll('(function(){var eds=document.querySelectorAll(".CodeMirror");' +
  'return eds.length>1&&eds[0].CodeMirror.getValue().length>0&&eds[1].CodeMirror.getValue(
  .length>0;})()',
  function (v) { return v === true; }, 60, '编辑器文本就绪');
await evaluate('(function(){var eds=document.querySelectorAll(".CodeMirror");' +
  'var L=eds[0].CodeMirror,R=eds[1].CodeMirror;' +
  'var rL=L.getValue(),rR=R.getValue();' +
  'window.__reportRawText__={L:rL,R:rR};' +      // 修整前原文（供报告"取消修整"）
  // 原始文本 diff：保留换行按行对比（ignoreNewline:false → grid 模式），其余选项与任务一致，
  // 供报告的"取消修整"视图显示带颜色标注 + 行号的原始文本 diff
  'var ro=' + JSON.stringify(rawOptions(M)) + ';' +
  'window.__reportRawDiff__=(typeof Compute!=="undefined"&&Compute.computeDiff)?' +
  'Compute.computeDiff({left:rL,right:rR,options:ro}):null;' +
  'L.setValue(Norm.tidyText(rL));R.setValue(Norm.tidyText(rR));return 1;})()');

// 显式再算一次（上面自动对比已按同样选项跑过，这里保证拿到确定的最新结果）
await evaluate('Pipeline.compare()');
var rpoll = await poll('(function(){var r=Pipeline.getResult();if(!r)return
  "{\\"done\\":false}";' +
  'if(r.error)return JSON.stringify({done:true,err:String(r.error).slice(0,300)});' +
  'return JSON.stringify({done:true,mode:r.mode});})()',
  function (v) { var s = JSON.parse(v); return s.done; }, 80, '对比结果');
var rinfo = JSON.parse(rpoll);
if (rinfo.err) throw new Error('对比失败：' + rinfo.err);
// 标注在 render() 内同步上色，但编辑器变更触发的防抖对比可能晚到，等它落定
await wait(1200);

// 诊断：docx 的 fitWidth 是否生效（scrollWidth≈面板宽 说明 zoom 已把页面收进面板，未被截断）
if (kindL === 'docx' || kindR === 'docx') {
  var zinfo = await evalJson('JSON.stringify((function(){var
  out={};["pdfLeft","pdfRight"].forEach(function(id){' +
    'var p=document.getElementById(id);var h=p.querySelector(".docx-host");if(!h)return;' +
    'var sec=p.querySelector(".docx-wrapper section.docx");' +
    'out[id]={zoom:h.style.zoom||"1",secW:sec?Math.round(sec.offsetWidth
  :0,panelW:p.clientWidth,scrollW:p.scrollWidth};});' +
    'return out;})())');
  log('  docx fitWidth：' + JSON.stringify(zinfo));
}

// 取结果（只取报告需要的字段，避免把超大 charAnchors 等一起拉回）
var result = await evalJson('(function(){var r=Pipeline.getResult();' +
  'if(!r)return "null";if(r.error)return JSON.stringify({error:r.error});' +
  'if(r.mode==="grid")return JSON.stringify({mode:"grid",rows:r.rows,leftLines:r.leftLines,' +
  'rightLines:r.rightLines,stats:r.stats});' +
  'if(r.mode==="flow"){if(r.skipped)return JSON.stringify({mode:"flow",skipped:true,leftLen:r.le
  ftLen,rightLen:r.rightLen});' +
  'return JSON.stringify({mode:"flow",segsL:r.segsL,segsR:r.segsR,addedChars:r.addedChars,remove
  dChars:r.removedChars});}' +
  'return "null";})()');

// 逐侧快照（面板里真正渲染的是什么就按什么截；PDF 优先直接光栅化，绕开合成器截图）
var kinds = await evalJson('(function(){function k(id){var p=document.getElementById(id);' +
  'if(p&&p.querySelector(".docx-host"))return "docx";if(p&&p.querySelectorAll(".pdf-page"
  .length)return "pdf";return "none";}' +
  'return JSON.stringify({L:k("pdfLeft"),R:k("pdfRight")});})()');
var shots = { L: [], R: [] };
if (kinds.L === 'docx') shots.L = await snapSide('L', 'pdfLeft', '.docx-wrapper section.docx',
  'L(docx)', false);
else if (kinds.L === 'pdf') shots.L = await snapSide('L', 'pdfLeft', '.pdf-page', 'L(pdf)',
  true);
if (kinds.R === 'docx') shots.R = await snapSide('R', 'pdfRight', '.docx-wrapper section.docx',
  'R(docx)', false);
else if (kinds.R === 'pdf') shots.R = await snapSide('R', 'pdfRight', '.pdf-page', 'R(pdf)',
```

```
    true);

  // 修整前原文（供报告"取消修整"按钮切换显示；无则 null）
  var rawText = await evalJson('JSON.stringify(window.__reportRawText__||null)');
  // 原始文本的 grid diff（保留换行，带颜色标注+行号）；Compute 不可用时为 null
  var rawDiff = await evalJson('JSON.stringify(window.__reportRawDiff__||null)');
  return { leftName: path.basename(left), rightName: path.basename(right), result: result, shots:
    shots,
    rawText: rawText, rawResult: rawDiff };
}

function addStats(stats, result) {
  if (!result) return;
  if (result.mode === 'grid' && result.stats) {
    stats.added += result.stats.added;
    stats.removed += result.stats.removed;
    stats.changed += result.stats.modified;
  } else if (result.mode === 'flow') {
    stats.added += result.addedChars || 0;
    stats.removed += result.removedChars || 0;
  }
}

// ---------- 主流程 ----------
(async function main() {
  if (!BROWSER) {
    process.stderr.write('找不到浏览器：请安装 Edge，或用 EDGE_PATH 指定浏览器可执行文件路径\n');
    process.stdout.write(JSON.stringify({ ok: false, output: null, stats: { pairs: 0, added: 0,
    removed: 0, changed: 0 },
      warnings: ['找不到可用的浏览器（Edge/Chrome）'] }) + '\n');
    process.exit(2);
  }
  var taskPath = parseArgs(process.argv);
  if (!taskPath) {
    process.stderr.write('用法：node tools/render.js --task <task.json>\n');
    process.stdout.write(JSON.stringify({ ok: false, output: null, stats: { pairs: 0, added: 0,
    removed: 0, changed: 0 },
      warnings: ['缺少 --task 参数'] }) + '\n');
    process.exit(1);
  }
  var task = JSON.parse(fs.readFileSync(taskPath, 'utf8'));
  if (!task || !Array.isArray(task.pairs) || !task.pairs.length) {
    process.stdout.write(JSON.stringify({ ok: false, output: null, stats: { pairs: 0, added: 0,
    removed: 0, changed: 0 },
      warnings: ['任务缺少非空 pairs 数组'] }) + '\n');
    process.exit(1);
  }
  // C3 分卷：一份报告默认最多 10 对，超出由调用方切成多份（本工具不自行分卷，直接拒绝）；
  // 上限优先级：任务字段 maxPairsPerReport > 环境变量 DSH_MAX_PAIRS_PER_REPORT > 10。
  // walk 模式"每文件夹一份报告"（volume:"folder"）时 dsh-report.js 会给该卷带上足够大的
  // maxPairsPerReport，使超 10 对的文件夹也能整份产出，不必拆卷。
  var maxPairs = parseInt(task.maxPairsPerReport || process.env.DSH_MAX_PAIRS_PER_REPORT || '10',
    10);
  if (!(maxPairs > 0)) maxPairs = 10;
  if (task.pairs.length > maxPairs) {
    var mCap = '一份报告最多 ' + maxPairs + ' 对（当前 ' + task.pairs.length + '
    对），请将任务拆成多份分别渲染';
    process.stderr.write(mCap + '\n');
    process.stdout.write(JSON.stringify({ ok: false, output: null, stats: { pairs: 0, added: 0,
    removed: 0, changed: 0 },
      warnings: [mCap] }) + '\n');
    process.exit(1);
  }
  var output = path.resolve(task.output || 'report.html');
```

```
fs.mkdirSync(path.dirname(output), { recursive: true });
var warnings = [];
var stats = { pairs: task.pairs.length, added: 0, removed: 0, changed: 0 };

// 临时静态服务
serverProc = cp.spawn(process.execPath, [path.join(ROOT, 'serve.js'), String(HTTP_PORT)], {
  stdio: 'ignore' });
var i, up = false;
for (i = 0; i < 40; i++) {
  try { await fetchJson('http://127.0.0.1:' + HTTP_PORT + '/api/list?path=' +
  encodeURIComponent(process.cwd())); up = true; break; }
  catch (e) { await wait(250); }
}
if (!up) throw new Error('serve.js 启动失败');

// headless Edge + CDP。沙箱环境（如 dsh 执行命令的沙箱）里不带 --no-sandbox 时，
// CDP 能连上（/json/list、WebSocket 均正常）但页面渲染进程不响应 Runtime.evaluate，
// 显式 Page.navigate 也可能不提交 → 一律带 --no-sandbox 启动（无头本地渲染的常规做法）。
// --force-device-scale-factor=2：无头下 devicePixelRatio=1，PdfView 按 dpr 光栅化画布
// 会把 PDF 页渲染成 1x → 文字发虚。强制 dpr=2 后画布/文字都按 2x 真实光栅化。
// 截图仍按 clip.scale=2（4x 像素）采集——dsf=2 下 scale=1 会踩 Chromium 确定性撕裂
//  （见 capture() 注释），4x 截回后 Node 内 box 降采样到 2x 成图，细节相同、体积 1/4。
function launchEdge(extraArgs) {
  var profile = path.join(os.tmpdir(), 'edge-render-' + process.pid + (extraArgs.length ? '-ns'
  : ''));
  edgeProc = cp.spawn(BROWSER, ['--headless=new', '--disable-gpu', '--no-first-run',
  '--hide-scrollbars',
    '--window-size=1600,1400', '--no-sandbox', '--force-device-scale-factor=2',
    '--remote-debugging-port=' + DBG_PORT, '--user-data-dir=' + profile]
    .concat(extraArgs, ['http://127.0.0.1:' + HTTP_PORT + '/']), { stdio: 'ignore' });
  return (async function () {
    for (var k = 0; k < 60; k++) {
      if (edgeProc.exitCode !== null) return null;   // 进程已死（如被沙箱拦），不必干等
      try {
        var list = await fetchJson('http://127.0.0.1:' + DBG_PORT + '/json/list');
        var t = (list || []).filter(function (x) { return x.type === 'page' &&
  x.url.indexOf('127.0.0.1:' + HTTP_PORT) !== -1; })[0];
        if (t) return t;
      } catch (e) { /* 未就绪 */ }
      await wait(300);
    }
    return null;
  })();
}
var pg = await launchEdge([]);
if (!pg) {
  log('浏览器首次启动未拿到页面目标，重试一次（兼容启动抖动）');
  try { if (edgeProc) edgeProc.kill(); } catch (e0) {}
  pg = await launchEdge(['--retry']);
}
if (!pg) throw new Error('未拿到浏览器页面目标');
ws = new WebSocket(pg.webSocketDebuggerUrl);
await new Promise(function (res, rej) { ws.onopen = res; ws.onerror = rej; });
ws.onmessage = function (ev) {
  var m = JSON.parse(ev.data);
  if (m.id && pending[m.id]) {
    var p = pending[m.id]; delete pending[m.id];
    m.error ? p.reject(new Error(m.error.message)) : p.resolve(m.result);
  }
};
log('CDP 已连接');

// ② 沙箱环境里初始导航可能从未提交（页面目标在但停在 about:blank），显式导航一次确保真正加载
try {
```

```
    await cdpSend('Page.navigate', { url: 'http://127.0.0.1:' + HTTP_PORT + '/' }, 15000);
    await wait(800);
  } catch (e) { log('  !! 显式导航失败（不致命，继续等页面就绪）: ' + e.message); }

  await poll('(typeof Pipeline!=="undefined")&&(typeof
    PdfView!=="undefined")&&!!document.querySelector(".CodeMirror")',
    function (v) { return v === true; }, 40, '页面脚本就绪');
  // 坑 2：先解除协同滚动并掐死重绑（布局变化会重绑回监听器，把 scrollTop 拉回 0）
  await evaluate('(function(){try{if(typeof SyncScroll!=="undefined"&&SyncScroll.rebind){' +
    'SyncScroll.rebind([]);SyncScroll.rebind=function(){return;};return "unbound+locked";}}' +
    'catch(e){return "err:"+e.message;}return "absent";})()');
  // 坑 4：注入无头布局，主区域 3 撑满窗口（必须在加载 docx 之前：fitWidth 按最终面板宽度算 zoom）
  await evaluate('(function(){var s=document.createElement("style");s.id="render-css";' +
    's.textContent="#results,#resultsResize,#editors,#editorsResize{display:none !important}"+' +
    '"#pdfarea{flex:1 1 auto !important;max-height:none !important;margin-top:0 !important}"+' +
    '"#pdfarea .pdf-controls{display:none !important}";document.head.appendChild(s);return
    1;})()');
  await wait(500);
  // 首屏 updatePdfArea() 会给 #pdfarea 加 hidden（无文件时不显示主区域3）；docx 的 fitWidth 在
  // load 时量面板宽算 zoom，此刻尚未有任何 onPanelsChanged 解除 hidden → clientWidth=0 → zoom
  // 退化为 1，页面按 A4 原大渲染、超出面板被截断。加载前先解除，与真实界面"有文件后显示主区域3"一
    致。
  await evaluate('(function(){var el=document.getElementById("pdfarea");if(el
    el.classList.remove("hidden");return 1;})()');

  // 逐对处理：单对失败只记 warning 继续
  var pairs = [];
  for (i = 0; i < task.pairs.length; i++) {
    armWatchdog();   // 看门狗按对重挂：大批量（>10 对）不会误杀
    try {
      var p = await renderPair(task.pairs[i], task.options, i, warnings);
      pairs.push(p);
      addStats(stats, p.result);
    } catch (e) {
      var msg = '第 ' + (i + 1) + ' 对失败: ' + ((e && e.message) || e);
      log('  !! ' + msg);
      warnings.push(msg);
      stats.pairs--;   // 成功的对数才计入
    }
  }
  if (!pairs.length) {
    process.stdout.write(JSON.stringify({ ok: false, output: null, stats: stats, warnings:
    warnings }) + '\n');
    process.exit(1);
  }

  // 组装报告并落盘
  var html = Report.build({
    title: task.title || '文本对比报告',
    time: new Date().toLocaleString('zh-CN'),
    pairs: pairs
  });
  fs.writeFileSync(output, html, 'utf8');
  log('报告已写入: ' + output + ' (' + (html.length / 1024).toFixed(0) + 'KB, ' + pairs.length +
    ' 对)');

  process.stdout.write(JSON.stringify({ ok: warnings.length === 0, output: output, stats: stats,
    warnings: warnings }) + '\n');
  process.exit(warnings.length ? 1 : 0);
})().catch(function (e) {
  process.stderr.write('FAIL  ' + ((e && e.message) || e) + '\n');
  try {
    process.stdout.write(JSON.stringify({ ok: false, output: null, stats: { pairs: 0, added: 0,
    removed: 0, changed: 0 },
```